AirfoilData

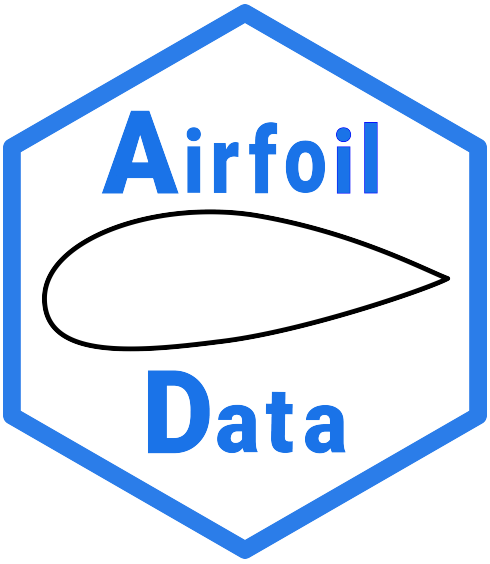
A C++ Library for Airfoil Data Storage, Analysis and Manipulation

User Manual

Olaf Frommann

Institute for Aerospace Technology

University of Applied Sciences Bremen



Version: 1.0.0
Date: October 2026
Repository: https://github.com/of33/AirfoilData

Contents

Revision History **3**

License **3**

1 Introduction **4**

1.1 Design Philosophy 4

1.2 Library overview 7

2 Coordinate and Unit Conventions **8**

2.1 Leading-Edge Detection 9

2.2 Angles 9

2.3 Finite Numerical Values 10

3 Theoretical Interpretation of the Stored Singularity Distributions **10**

3.1 Scaled Source–Sink Distribution 10

3.2 Normalized Circulation Distribution 11

3.3 Purpose and Limitations 13

4 Requirements and Installation **13**

4.1 Required Software 13

4.2 Obtaining a Release Version 14

4.3 Installing the Required Tools on Windows 15

4.4 Installing the Required Tools on Linux 18

4.5 Installing the Required Tools on macOS 19

5 Quick Start **20**

6 Basic, Advanced and Integration API **22**

6.1 Basic API 22

6.2 Advanced API 22

6.3 Integration API 23

7 Example Program **23**

7.1 Operations Performed by the Example 23

7.2 Example Console Output 24

7.3 Generated Data Files 25

7.3.1 `example_airfoil.dat` 25

7.3.2 `example_camber.dat` 25

7.3.3 `example_circ_source.dat` 25

7.3.4 `example_grad_curv.dat` 26

8 Plot Generation with Gnuplot **26**

8.1 Airfoil and Thickness Distribution 27

8.2 Camber Line 27

8.3 Circulation and Source Strength 28

8.4 Surface Derivatives 29

9 Building and Testing **29**

9.1 Manual Compilation of the Core Object Files 30

9.2 Building the Example 31

9.3 Building the Core Object Files with Make 31

9.4 Optional Static Archive 32

9.5 Running the Example 32

9.6 Running the Standard Test Set 33

9.7 Skipped Optional Checks 33

9.8 Test-Harness Design 34

9.9 Individual Test Targets . . . 35
9.10 Updating the Golden Reference . . . 36
10 Akima Interpolation **37**
10.1 Input Requirements . . . 37
10.2 Basic Example . . . 37
10.3 Available Results . . . 38
10.4 Recommended Akima Workflow . . . 39
10.5 Interpolating an Airfoil Camber Line . . . 39
10.6 AkimaInterpolation API Reference . . . 41
10.7 Writing an Interpolated Data Table . . . 45
10.8 Recommendations for Akima Interpolation . . . 47
10.9 Akima Interpolation Troubleshooting . . . 47
11 Public Interface Overview **48**
11.1 Recommended Basic Workflow . . . 48
12 Public Types **49**
12.1 `NoseDetectionMode` . . . 49
12.2 `ThicknessMode` . . . 50
12.3 `XDistributionMode` . . . 51
12.4 `ThicknessJob` . . . 51
12.5 `SideMode` . . . 51
12.6 `CamberDifferenceMode` . . . 52
12.7 `SortMode` . . . 52
12.8 `InterpolationMode` . . . 52
12.9 `RedistributeMode` . . . 53
12.10 `RecalcMode` . . . 53
12.11 `TrailingEdgeCheck` . . . 53
12.12 `AeroScalars` . . . 54
12.13 `AirfoilDataMessageType` . . . 55
12.14 `GradientData` . . . 55
13 Basic API Reference **56**
13.1 Construction and Reset . . . 56
13.2 Airfoil Name and Coordinates . . . 57
13.3 Airfoil Validation . . . 59
13.4 Complete Property Calculation . . . 59
13.5 Airfoil Redistribution . . . 60
13.6 Principal Scalar Results . . . 61
13.7 Calculated Distributions . . . 62
13.8 Saving Airfoil Coordinates . . . 64
13.9 Version Information . . . 65
14 Advanced API Reference **66**
14.1 Support-Point Management . . . 66
14.2 Geometry Preparation and Transformation . . . 69
14.3 Camber, Thickness, and Area Calculations . . . 76
14.4 Surface Separation and Derivatives . . . 82
14.5 Normalized Vortex-Sheet and Scaled Source–Sink Calculations . . . 84
14.6 Advanced Workflow Examples . . . 89
15 Integration API Reference **91**
15.1 Diagnostic Output and Callbacks . . . 91

15.2 Application-Level Object State . . . 95
15.3 External Aerodynamic Data . . . 96
15.4 Pressure-Coefficient Data . . . 99
15.5 Low-Level Geometry Setters . . . 100
15.6 Imported Derivative Data . . . 102
15.7 Conformal-Mapping Data . . . 103
15.8 Integration API Recommendations . . . 104
16 Development and Contributing 105
16.1 Design Principles for Contributions . . . 105
16.2 Source and Test Layout . . . 108
16.3 Test-Harness Architecture . . . 108
16.4 Exact Serialization and Tolerance-Based Comparison . . . 109
16.5 Scenario Coverage . . . 110
16.6 Running the Tests During Development . . . 111
16.7 Adding a Regression Scenario . . . 113
16.8 Capturing Library Output . . . 114
16.9 Updating the Golden Reference . . . 114
16.10 Numerical Changes . . . 115
16.11 Public API Changes . . . 116
16.12 Coding Conventions . . . 116
16.13 Documentation and Theory . . . 117
16.14 Recommended Contribution Workflow . . . 117
16.15 Contribution Checklist . . . 118
References 119

Revision History

Revision	Date	Type	Changes
1.0.0	October 2026	Initial release	Initial public version.

License

Copyright (C) 2026 Olaf Frommann

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see https://www.gnu.org/licenses/.

1 Introduction

AirfoilData is a C++17 library for storing, validating, normalizing, redistributing, and analyzing two-dimensional airfoil coordinates. It is intended for engineering and scientific applications and provides both a high-level workflow for common airfoil operations and an extensive public interface for specialized analyses and application integration.

The library has a small integration footprint and a deliberately readable implementation. Ordinary use requires only the AirfoilData.h interface and the two compiled object files AirfoilData.o and AkimaInterpolation.o. No static-library archive, shared-library installation, generated source code, or external numerical package is required. The complete source implementation consists of four core files and depends only on the C++ standard library.

1.1 Design Philosophy

AirfoilData is intended to be a practical engineering library rather than a general-purpose numerical framework. The design is informed by several decades of professional experience with numerical, aerodynamic, and engineering software.

The principal design goals are:

- simple use and integration,
- readable numerical source code,
- direct applicability to practical airfoil work,
- a small number of source and interface files,
- minimal external dependencies,
- explicit and predictable error handling,
- high numerical and memory reliability,
- reproducible behavior supported by an extensive regression harness.

Simple Integration

The complete library consists of four core files:

```
src/AirfoilData.h
src/AirfoilData.cpp
src/AkimaInterpolation.h
src/AkimaInterpolation.cpp
```

An application using the main airfoil interface normally includes only:

```
#include "AirfoilData.h"
```

By default, the supplied Makefile compiles the two implementation files into ordinary object files:

```
build/AirfoilData.o
build/AkimaInterpolation.o
```

These object files are linked directly into the application. A static or shared library is not required.

The object files can be generated explicitly with:

```
make objects
```

Applications that use the standalone interpolation class directly additionally include:

```
#include "AkimaInterpolation.h"
```

An application using only `AkimaInterpolation` needs to link only `AkimaInterpolation.o`.

On Linux and macOS, the object files can optionally be collected in a static archive for convenience:

```
make lib
```

This creates:

```
lib/libAirfoilData.a
```

The archive contains the same two object modules used by the default direct linking process. It does not change the public interface or add external dependencies.

Note

C++ object files and static-library archives are specific to the compiler toolchain, ABI, target architecture, and relevant build options. They should therefore be generated with a toolchain compatible with the application.
For reproducible and portable builds, compile the object files locally from the source release.

Readability Before Abstraction

The source code is intended to remain understandable to engineers, scientists, students, and other users who are not primarily software specialists.

For this reason, the numerical implementation favors:

- explicit variable types,
- descriptive names,
- conventional loops and control structures,
- visible intermediate calculation steps,
- strongly typed `enum class` values instead of numerical flags,
- comments explaining numerical and aerodynamic reasons,
- a direct correspondence between mathematical operations and their C++ implementation.

Abstraction is used where it improves safety or removes genuine duplication, but not merely to shorten the source code. A slightly longer implementation is preferred when it makes the numerical method easier to inspect, verify, and maintain.

This does not imply that modern C++ language features are generally undesirable. The library uses appropriate C++17 facilities where they provide a clear practical benefit. However, new language constructs should not obscure the engineering procedure they implement.

Practical Engineering Use

The library focuses on operations required in real airfoil-processing workflows, including:

- validation and normalization of coordinate data,
- redistribution onto suitable support points,
- extraction of camber and thickness distributions,
- calculation of geometric properties,
- derivative evaluation,
- generation of normalized singularity distributions,
- storage and exchange of externally calculated aerodynamic data.

The public interface is intentionally extensive. The Basic API provides a short path for ordinary applications, while the Advanced and Integration APIs expose individual processing stages for expert users and larger software systems.

Defensive Reliability

Numerical engineering software often receives data from files, measurement systems, geometry generators, iterative solvers, or external applications. Such data can be incomplete, inconsistent, non-finite, or numerically extreme.

The implementation therefore applies defensive checks where they are relevant, including checks for:

- inconsistent vector dimensions,
- insufficient support-point counts,
- invalid point-count ranges,
- duplicated or incorrectly ordered interpolation abscissas,
- NaN and positive or negative infinity,
- non-finite intermediate and final results,
- zero or non-representable scaling intervals,
- aliased output parameters where distinct outputs are required,
- invalid leading-edge or contour geometry,
- failed file operations,
- invalid interpolation state.

Where practical, calculations are performed first in local temporary variables or vectors. Object members and output vectors are replaced only after the complete operation has succeeded. This transactional approach reduces the risk of leaving an object in a partially updated state.

The library also uses standard-library containers and value semantics to avoid manual memory ownership in the public numerical classes.

Note

Defensive validation cannot make arbitrary numerical input meaningful, and no software can guarantee the absence of every defect. The purpose of these checks is to detect common data, memory, state, and floating-point problems as early and predictably as possible. Applications must still check returned status values and respect the documented input requirements.

Verification and Reproducibility

Reliability is supported by a regression harness that exercises the public interface with ordinary, boundary, and invalid inputs. The developer checks include:

- golden-reference comparisons,
- GCC and Clang builds,
- strict compiler warnings,
- address and undefined-behavior sanitizers,
- debug standard-library checks,
- method coverage,
- static analysis,
- Valgrind memory checks.

The tests are not treated as a substitute for understandable source code. Instead, readable implementation, documented numerical conventions, input validation, and automated verification are intended to complement one another.

The remainder of this manual follows the same philosophy. It first presents the coordinate conventions and the preferred Basic API workflow, then documents the more specialized Advanced and Integration APIs. The final developer section explains how these design principles are preserved when the library is modified or extended.

1.2 Library overview

The library can calculate, among other quantities:

- normalization of airfoil contours,
- redistribution onto new support points,
- maximum airfoil thickness and its chordwise location,
- maximum camber and its chordwise location,
- trailing-edge thickness,
- cross-sectional area,
- a characteristic nose thickness,
- upper and lower surface coordinates,
- camber and thickness distributions,
- surface gradients and second chordwise derivatives,
- normalized vortex-sheet and scaled source–sink distributions,
- the inviscid zero-lift angle and pitching-moment coefficient.

The accompanying class `AkimaInterpolation` implements one-dimensional Akima interpolation for strictly ascending support points. It provides the interpolated value and, optionally, the first and second derivatives.

The library is designed for users with different levels of programming experience. The documentation therefore distinguishes between three parts of the public interface:

Basic API
: The methods required for loading coordinate data into an `AirfoilData` object, checking the contour, calculating the main properties, redistributing the airfoil, and retrieving the results.

Advanced API
: Methods that expose individual numerical processing steps, including normalization, geometric transformations, support-point generation, derivative calculation, camber and thickness extraction, and calculation of normalized vortex-sheet and scaled source–sink distributions.

Integration API
: Methods intended for embedding `AirfoilData` into a larger application. These include diagnostic output callbacks, external aerodynamic and pressure data, application-level object state, and low-level setters for individual intermediate data fields.

A typical basic workflow is:

1. read the airfoil coordinates into two `std::vector<double>` objects,
2. create an `AirfoilData` object,
3. assign the coordinate vectors,
4. check the airfoil contour,
5. calculate the airfoil properties,
6. retrieve the required geometric or aerodynamic quantities,
7. optionally redistribute the airfoil to a different number of support points.

Most operations that can fail return `bool`. A return value of `true` indicates success. A return value of `false` indicates invalid input data, inconsistent object data, or a numerical failure. Return values marked with `[[nodiscard]]` should always be checked.

The library does not depend on Qt or another application framework. It uses only the C++ standard library and can therefore be integrated into console programs, desktop applications, and numerical engineering software.

Developers who intend to modify or extend the library should consult Section 16, which describes the regression harness, numerical test strategy, and contribution workflow.

2 Coordinate and Unit Conventions

Many operations in `AirfoilData` assume a conventional two-dimensional airfoil contour.

For a normalized airfoil, the chordwise coordinate is:

$$0 \leq x \leq 1,$$

where $x = 0$ denotes the leading edge and $x = 1$ denotes the trailing edge. The vertical coordinate y is expressed relative to the chord length.

The usual contour order is:

1. lower trailing edge,
2. lower surface towards the leading edge,
3. leading edge,
4. upper surface towards the upper trailing edge.

Consequently, the x coordinates normally decrease on the lower branch and increase on the upper branch.

The leading-edge reference used during normalization is determined from the longest secant between an interior contour point and the midpoint of the two contour endpoints. This definition supports both closed and open trailing edges.

The trailing edge may be geometrically closed or may have a finite thickness. Methods that distinguish these cases preserve a finite trailing-edge thickness where possible.

2.1 Leading-Edge Detection

Depending on the processing step, the library determines the leading edge using one of two methods.

With `NoseDetectionMode::LocalMinimum`, the leading edge is determined from a local minimum of the chordwise coordinate. This mode is primarily used for contours that have already been normalized and oriented.

With `NoseDetectionMode::LongestSecant`, the secant origin is the midpoint of the two contour endpoints:

$$x_{\mathrm{TE}} = x_0 + \frac{1}{2}(x_{n-1} - x_0), \qquad y_{\mathrm{TE}} = y_0 + \frac{1}{2}(y_{n-1} - y_0).$$

The leading-edge candidate is the interior contour point with the greatest Euclidean distance from this trailing-edge midpoint:

$$i_{\mathrm{LE}} = \underset{1 \le i \le n-2}{\arg\max} \sqrt{(x_i - x_{\mathrm{TE}})^2 + (y_i - y_{\mathrm{TE}})^2}.$$

For a closed trailing edge, the first and last contour points coincide and the midpoint is identical to the trailing-edge point. For an open trailing edge, the midpoint lies between the lower and upper trailing-edge points. Using this midpoint avoids a bias towards either trailing-edge corner and provides the chord reference required for profiles with finite trailing-edge thickness.

The distance calculation uses `std::hypot` when a direct sum of squares could overflow.

2.2 Angles

All geometric and aerodynamic angles interpreted or calculated by the public numerical methods of `AirfoilData` are expressed in degrees:

- `rotate` receives the rotation angle in degrees,
- `normalizeAirfoil` reports the performed rotation in degrees,
- `calculateCirculation` receives the angle of attack in degrees,
- `getAlpha0Geo` returns the inviscid zero-lift angle in degrees.

Internally, trigonometric calculations use radians as required by the C++ standard-library functions. The necessary conversion is performed by the library.

The sign convention of `rotate` follows the implemented coordinate transformation: a positive angle rotates the coordinates clockwise in the conventional Cartesian coordinate system.

Imported auxiliary data that are merely stored by the Integration API retain the unit convention of the supplying application unless a specific unit is documented. Degrees are recommended for imported aerodynamic angle data.

2.3 Finite Numerical Values

Coordinate values and numerical method parameters must be finite. NaN and positive or negative infinity are rejected by the main calculation methods.

Very small distances between adjacent support points can nevertheless produce large gradients. Input coordinate sets should therefore avoid unnecessarily close or numerically indistinguishable support points.

3 Theoretical Interpretation of the Stored Singularity Distributions

`AirfoilData` calculates two distributions derived from classical linearized airfoil theory:

- a thickness-related source–sink distribution,
- a camber-related vortex-sheet or circulation distribution.

These distributions are used primarily as geometry descriptors and as input for airfoil-generation methods. They must therefore be distinguished from dimensional physical singularity strengths.

The following subsections define the normalization and reference condition used by the library. A detailed derivation of the underlying theory is given in the companion reports on the thickness and camber-line formulations of thin-airfoil theory [1, 2].

3.1 Scaled Source–Sink Distribution

In the thickness formulation of linearized airfoil theory, a symmetric airfoil is represented by its half-thickness distribution $z^{(t)}(x)$ and a source–sink distribution $q(x)$ on the chord line.

The linearized kinematic boundary condition gives:

$$q(x) = 2U_\infty \frac{dz^{(t)}}{dx}.$$

Consequently,

$$\frac{q(x)}{U_\infty} = 2\frac{dz^{(t)}}{dx}.$$

The method `calculateSourceStrength` does not store the dimensional source strength q and does not store the complete normalized quantity q/U_∞. Instead, it stores the scaled geometric distribution

$$q^*(x) = \frac{q(x)}{2U_\infty} = \frac{dz^{(t)}}{dx}.$$

Thus, the values returned by `getSourceY` are the chordwise derivatives of the one-sided half-thickness distribution.

For a normalized airfoil with x/c and $z^{(t)}/c$, this derivative is dimensionless:

$$q^* = \frac{d(z^{(t)}/c)}{d(x/c)}.$$

The corresponding physical or normalized source strength can be recovered when the free-stream velocity is known:

$$q = 2U_\infty q^*, \qquad \frac{q}{U_\infty} = 2q^*.$$

The factor of two is deliberately omitted in the stored distribution because the present library uses the result primarily as a geometric descriptor for a source-based airfoil generator. A constant scaling factor does not change the characteristic chordwise shape, zero crossings, or relative source and sink regions.

Positive values of q^* correspond to increasing half thickness and therefore to source regions. Negative values correspond to decreasing half thickness and therefore to sink regions.

For a closed half-thickness distribution satisfying

$$z^{(t)}(0) = z^{(t)}(c) = 0,$$

the scaled distribution satisfies the closure relation

$$\int_0^c q^*(x)\,dx = z^{(t)}(c) - z^{(t)}(0) = 0.$$

For the normalized coordinates used by the library, the equivalent relation is

$$\int_0^1 \frac{d(z^{(t)}/c)}{d(x/c)}\,d(x/c) = \frac{z^{(t)}(c) - z^{(t)}(0)}{c} = 0.$$

Multiplication by $2U_\infty$ gives the corresponding physical closure condition:

$$\int_0^c q(x)\,dx = 0.$$

Note

Despite the API names `calculateSourceStrength`, `getSourceY`, and `source strength`, the stored values are the scaled distribution

$$q^* = q/(2U_\infty) = dz^{(t)}/dx.$$

They are not dimensional source strengths.

3.2 Normalized Circulation Distribution

In the camber-line formulation of thin-airfoil theory, the airfoil is represented by a vortex sheet of strength $k(x)$ on the chord line.

Using the angular coordinate

$$\frac{x}{c} = \frac{1}{2}\left(1 + \cos\varphi\right),$$

the normalized vortex-sheet strength is represented as

$$\frac{k(\varphi)}{U_\infty} = 2\left[A_0 \tan\left(\frac{\varphi}{2}\right) + \sum_{n=1}^{N} A_n \sin(n\varphi)\right].$$

The method `calculateCirculation` evaluates this normalized quantity. Therefore, the values returned through `circulationStrength` and stored in `getCirculationY` are interpreted as

$$k^*(x) = \frac{k(x)}{U_\infty}.$$

They are not dimensional circulation densities. If a dimensional vortex-sheet strength is required, it can be reconstructed from

$$k(x) = U_\infty k^*(x).$$

Unlike the source distribution, no additional constant factor is removed from the theoretical normalized vortex-sheet expression.

The standard property calculation does not store the distribution at a fixed angle of attack. Instead, it uses the reference lift coefficient

$$c_l = 0.5.$$

According to linearized thin-airfoil theory,

$$c_l = 2\pi(\alpha - \alpha_0),$$

where the angles are expressed in radians in this equation. The angle of attack used for the reference distribution is therefore

$$\alpha_{c_l=0.5} = \alpha_0 + \frac{0.5}{2\pi}.$$

The library first determines the geometric zero-lift angle α_0 from the camber line. It then calculates the normalized circulation distribution at the angle corresponding to $c_l = 0.5$.

This fixed-lift reference makes circulation distributions from different camber lines more directly comparable and provides a standardized input for a circulation-based airfoil generator. Such a generator is not part of the present library.

Note

The standard circulation distribution is calculated at $c_l = 0.5$, not at an angle of attack of 0.5 degrees or 0.5 radians.

3.3 Purpose and Limitations

The two stored distributions have related but different normalizations:

Stored distribution	Library value	Theoretical relation
Source–sink distribution	$q^* = \dfrac{dz^{(t)}}{dx}$	$q = 2U_\infty q^*$
Circulation distribution	$k^* = \dfrac{k}{U_\infty}$	$k = U_\infty k^*$

Both distributions are based on linearized, two-dimensional, incompressible, inviscid airfoil theory. They should not be interpreted as complete viscous flow solutions or as replacements for panel, Euler, RANS, or experimental analysis.

In `AirfoilData`, their principal purpose is to characterize the geometry in a form suitable for subsequent geometry-generation methods:

- the scaled source distribution describes the thickness contribution,
- the normalized circulation distribution describes the camber and lift contribution at the standard reference lift coefficient.

For the complete theoretical derivations, sign conventions, coordinate transformation, Fourier representation, and numerical examples, see the companion reports [1, 2].

4 Requirements and Installation

4.1 Required Software

To compile and run the example program, the following software is required:

- a C++17-compatible compiler,
- GNU Make or a compatible `make` implementation.

The following additional tools are recommended:

- `gnuplot` for generating the documented PNG plots,
- `git` for cloning the repository and contributing changes.

The repository can alternatively be downloaded as a ZIP archive using a web browser, in which case `git` is not required.

All required tools are freely available and can be installed without additional license costs. The installation commands below use commonly available open source software packages.

The library itself consists of the four files:

- `AirfoilData.h`
- `AirfoilData.cpp`
- `AkimaInterpolation.h`
- `AkimaInterpolation.cpp`

The inclusion and compilation of these files are explained in Sections 5 and 9. The two header files provide the public declarations. The two implementation files can either be compiled directly together with an application or compiled once into the object files:

```
AirfoilData.o
AkimaInterpolation.o
```

No archive tool is required for the default direct-object build. The optional `make lib` target additionally requires the archive tool `ar`, which is normally provided together with GCC/binutils or the macOS command line tools.

4.2 Obtaining a Release Version

For ordinary use, it is recommended to download a numbered release of `AirfoilData` from the GitHub Releases page rather than downloading the current development state of the repository.

The release page is available at:

https://github.com/of33/AirfoilData/releases

On this page, select either the latest release or a specific version required for reproducibility. Each release provides a source-code archive, usually named `Source code (zip)` or `Source code (tar.gz)`.

For example, to use version 1.0.0, download the corresponding release archive from:

https://github.com/of33/AirfoilData/releases/tag/v1.0.0

After downloading, extract the archive to a directory of your choice. Using a release archive ensures that the source code, examples, tests, documentation, and changelog belong to the same documented version.

Note

For reproducible scientific or engineering work, use a specific numbered release instead of the current development branch. If a Zenodo version-specific DOI is available for the release, cite that DOI for exact reproducibility. The Zenodo Concept DOI identifies the overall software project, while version-specific DOIs identify individual archived releases.

Using the Latest Release

If no specific version is required, use the latest release listed on the GitHub Releases page:

https://github.com/of33/AirfoilData/releases/latest

This redirects to the most recent published release.

Using Git for Development

Cloning the repository with `git` is mainly recommended for development and contributing to the project:

```
git clone https://github.com/of33/AirfoilData.git
cd AirfoilData
```

By default, this obtains the current state of the default branch, which may differ from the latest release. To work with a specific release tag, first list the available tags:

```
git tag
```

For example, for version 1.0.0 the tag is expected to be `v1.0.0`, unless stated otherwise on the release page. Then check out the desired tag, for example:

```
git checkout <release-tag>
```

If the tag name differs from this pattern, use the exact tag shown on the GitHub release page.

Note

When using `git checkout` on a release tag, Git may enter a so-called "detached HEAD" state. This is normal when inspecting or building a fixed release version. For development work, create a branch before making changes.

4.3 Installing the Required Tools on Windows

On Windows, it is recommended to use MSYS2. MSYS2 provides a modern package manager, Unix-like command line tools, and up-to-date GCC compiler toolchains for building native Windows programs.

Download and install MSYS2 from:

https://www.msys2.org/

Opening the MSYS2 UCRT64 Terminal

After installation, open an MSYS2 terminal. On Windows, this can be done from the Start menu. Search for `MSYS2` and start the entry named `MSYS2 UCRT64`. This terminal provides a suitable environment for compiling native Windows executables with GCC.

MSYS2 uses the package manager `pacman`. It is used to update the system and to install additional software packages.

First update the package database and the core MSYS2 packages:

```
pacman -Syu
```

Here, `-S` means "synchronize packages", `-y` refreshes the package database, and `-u` upgrades installed packages.

If requested, close the terminal window after the update has finished. This is sometimes necessary after core system packages have been updated. Then open `MSYS2 UCRT64` again from the Start menu and continue the update with:

```
pacman -Su
```

This upgrades the remaining installed packages using the already refreshed package database.

Install the compiler, Make, gnuplot, and git:

```
pacman -S --needed git make mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-gnuplot
```

The option `-needed` tells `pacman` to skip packages that are already installed and up to date. The preceding packages are sufficient for building the library object files, the example program, and the documented plots. The additional Clang packages described below are needed only for the corresponding developer test targets.

The installation can be checked with:

```
g++ --version
make --version
gnuplot --version
git --version
```

If one of these commands is not found, make sure that the `MSYS2 UCRT64` terminal is used. Other MSYS2 terminals, such as the plain `MSYS` terminal, may not provide the same compiler environment by default.

The examples should be built from the `MSYS2 UCRT64` terminal, not from the standard Windows command prompt.

Accessing the AirfoilData Directory from MSYS2

After opening the `MSYS2 UCRT64` terminal, the user is inside a Unix-like shell environment. This environment uses Unix-style paths. Windows drives are mounted below `/c`, `/d`, and so on.

For example, the Windows directory

```
C:\Users\Alice\Downloads\AirfoilData
```

is accessed in MSYS2 as:

```
/c/Users/Alice/Downloads/AirfoilData
```

To enter this directory, run:

```
cd /c/Users/Alice/Downloads/AirfoilData
```

The current directory can be displayed with:

```
pwd
```

The files in the current directory can be listed with:

```
ls
```

Note

The MSYS2 terminal uses a Unix-like shell. Commands, file names, directory names, and Makefile targets should be treated as case-sensitive. For example, `make plot` is not the same as `make Plot`, and `airfoildata.h` is not the same spelling as `AirfoilData.h`. Using the exact capitalization shown in this manual is recommended. This also improves portability to Linux and macOS, where file names are commonly case-sensitive.

Before running `make`, check that the current directory contains the project files, for example:

```
src/
    AirfoilData.h
    AirfoilData.cpp
    AkimaInterpolation.h
    AkimaInterpolation.cpp
Makefile
example.cpp
```

If the repository has not yet been downloaded, it can also be cloned directly from the `MSYS2 UCRT64` terminal:

```
git clone https://github.com/of33/AirfoilData.git
cd AirfoilData
```

Optional Developer Test Tools

Building the core object files and the example program requires only the GCC toolchain and GNU Make. Gnuplot is additionally required for generating the documented plots.

The standard developer check set also uses Clang and clang-tidy where available. Install the corresponding MSYS2 UCRT64 packages with:

```
pacman -S --needed \
mingw-w64-ucrt-x86_64-clang \
mingw-w64-ucrt-x86_64-clang-tools-extra
```

Check the installation with:

```
clang++ --version
clang-tidy --version
```

These packages are not required for:

- using `AirfoilData` in another application,
- building the core object files,
- creating the optional static archive,
- building or running the example,
- generating the numerical example data,

- generating the plots when gnuplot is already installed.

The Clang packages provide the compiler and clang-tidy required by the corresponding compiler and static-analysis targets. Depending on the current MSYS2 package set, the sanitizer runtime `compiler-rt` may still be unavailable.

Similarly, the installed GCC toolchain may not provide usable `libasan` and `libubsan` runtime libraries. The test Makefile detects unavailable sanitizer runtimes, skips the affected optional target, and prints a warning.

Note

Install these packages and run the tests from the `MSYS2 UCRT64` terminal. Packages installed for another MSYS2 environment, such as MINGW64 or CLANG64, are not necessarily available in the UCRT64 environment.

4.4 Installing the Required Tools on Linux

On Debian-based systems such as Ubuntu, install the required packages with:

```
sudo apt update
sudo apt install g++ make gnuplot git
```

On Fedora, use:

```
sudo dnf install gcc-c++ make gnuplot git
```

On Arch Linux, use:

```
sudo pacman -S gcc make gnuplot git
```

The installation can be checked with:

```
g++ --version
make --version
gnuplot --version
git --version
```

Accessing the AirfoilData Directory on Linux

If the repository has not yet been downloaded, it can be cloned with:

```
git clone https://github.com/of33/AirfoilData.git
cd AirfoilData
```

If the repository was downloaded as a ZIP archive, extract it and change into the extracted directory, for example:

```
cd ~/Downloads/AirfoilData
```

Use `pwd` to display the current directory and `ls` to list its contents.

Before running `make`, check that the current directory contains the project files:

```
src/
    AirfoilData.h
    AirfoilData.cpp
    AkimaInterpolation.h
    AkimaInterpolation.cpp
Makefile
example.cpp
```

4.5 Installing the Required Tools on macOS

On macOS, install the Xcode command line tools:

```
xcode-select --install
```

This provides a C++ compiler, `make`, `git`, and other basic development tools.

For installing `gnuplot`, Homebrew is recommended. If Homebrew is not installed yet, follow the instructions at:

https://brew.sh/

Then install gnuplot:

```
brew install gnuplot
```

Check the installation with:

```
clang++ --version
make --version
gnuplot --version
git --version
```

Accessing the AirfoilData Directory on macOS

If the repository has not yet been downloaded, it can be cloned with:

```
git clone https://github.com/of33/AirfoilData.git
cd AirfoilData
```

If the repository was downloaded as a ZIP archive, extract it and change into the extracted directory, for example:

```
cd ~/Downloads/AirfoilData
```

Use `pwd` to display the current directory and `ls` to list its contents.

Before running `make`, check that the current directory contains the project files:

```
src/
    AirfoilData.h
    AirfoilData.cpp
```

```
    AkimaInterpolation.h
    AkimaInterpolation.cpp
Makefile
example.cpp
```

5 Quick Start

This section shows the central workflow of the `AirfoilData` library. It assumes that the airfoil coordinates have already been read into the vectors `x` and `y`.

```cpp
#include "AirfoilData.h"

#include <iostream>
#include <vector>

int main()
{
    std::vector<double> x;
    std::vector<double> y;

    // Read the airfoil coordinates into x and y here.

    AirfoilData airfoil;
    airfoil.setAirfoilName("Example airfoil");

    if (!airfoil.setAirfoilPoints(x, y))
    {
        std::cerr
        << "The coordinate vectors are inconsistent.\n";
        return 1;
    }

    if (!airfoil.airfoilOK(TrailingEdgeCheck::Full))
    {
        std::cerr << "The airfoil contour is not valid.\n";
        return 1;
    }

    if (!airfoil.calculateAirfoilProperties())
    {
        std::cerr
        << "Could not calculate airfoil properties.\n";
        return 1;
    }

    std::cout << "Thickness: "
    << airfoil.getThickness() << "\n";

    std::cout << "Thickness location: "
    << airfoil.getThicknessLocation() << "\n";

    std::cout << "Camber: "
    << airfoil.getCamber() << "\n";

    std::cout << "Camber location: "
```

```
46      << airfoil.getCamberLocation() << "\n";
47
48      std::cout << "Area: "
49      << airfoil.getArea() << "\n";
50
51      std::cout << "Zero-lift angle: "
52      << airfoil.getAlpha0Geo()
53      << " deg\n";
54
55      return 0;
56  }
```

The method `calculateAirfoilProperties` performs the complete standard calculation pipeline. It calculates the camber and thickness distributions, the principal geometric properties, normalized vortex-sheet and scaled source–sink distributions, upper and lower surface data, and gradient and curvature data.

All returned values should be used only after the method has returned `true`.

The program can be compiled directly together with the two implementation files:

```
g++ -std=c++17 -O2 -Wall -Wextra -Isrc \
main.cpp \
src/AirfoilData.cpp \
src/AkimaInterpolation.cpp \
-o main
```

Alternatively, use the reusable object files produced by `make objects`, or the optional static archive produced by `make lib`; see Section 9.

On Linux and macOS, run the program with:

```
./main
```

On Windows, the executable may be called:

```
main.exe
```

The compiled library version can be queried with:

```
1  std::cout << "AirfoilData version "
2  << AirfoilData::versionString()
3  << "\n";
```

Note

`setAirfoilPoints` is the preferred method for assigning a complete airfoil contour. It checks that the two coordinate vectors have equal lengths before replacing the stored contour.
A successful call clears all previously derived geometry, circulation, source-strength, gradient, external aerodynamic, and pressure data. It also resets the created state to `false`.
The method does not perform complete geometric validation. Use `airfoilOK` before calculating the complete airfoil properties.

6 Basic, Advanced and Integration API

The complete public interface is intentionally extensive. It supports both ordinary airfoil-processing tasks and specialized engineering applications. For easier orientation, this manual classifies the public interface into three levels: the Basic API, the Advanced API, and the Integration API.

6.1 Basic API

The Basic API contains the methods normally required for complete airfoil processing.

Method	Purpose
`setAirfoilName`	Assigns a descriptive name to the airfoil.
`setAirfoilPoints`	Assigns both airfoil coordinate vectors after checking that their dimensions agree. This is the preferred coordinate input method.
`airfoilOK`	Checks whether the contour represents a usable airfoil.
`calculateAirfoilProperties`	Performs the complete standard property calculation.
`redistributeAirfoil`	Redistributes the contour to a requested odd number of support points.
`getThickness`, `getThicknessLocation`	Return the maximum thickness and its location.
`getCamber`, `getCamberLocation`	Return the maximum camber and its location.
`getArea`	Returns the calculated cross-sectional area.
`getTrailingEdgeThickness`	Returns the trailing-edge thickness.
`getAirfoilX`, `getAirfoilY`	Return the current contour coordinates.
`getCamberLineX`, `getCamberLineY`	Return the calculated camber line.
`saveAirfoilCoordinates`	Writes the current airfoil coordinates to a file.

6.2 Advanced API

The Advanced API contains methods for:

- support-point generation and management,
- direct scaling, rotation, and normalization,
- leading-edge detection,
- mapping onto new support points,
- repair of small chordwise contour reversals,
- separate calculation of thickness, area, and nose properties,
- extraction of camber and thickness distributions,
- separation of upper and lower surfaces,
- derivative calculations,
- normalized vortex-sheet and scaled source–sink distributions.

The Advanced API is intended for users who require direct control over individual numerical processing steps.

6.3 Integration API

The Integration API contains methods for:

- diagnostic output callbacks and output suppression,
- diagnostic verbosity control,
- application-level created state,
- storage of externally calculated aerodynamic data,
- storage of pressure-coefficient distributions,
- storage of conformal-mapping and imported derivative data,
- direct low-level replacement of individual geometry fields.

The Integration API is intended primarily for application adapters, graphical user interfaces, external solvers, and specialized data-import workflows.

7 Example Program

The distribution contains the example program:

```
example.cpp
```

The program demonstrates a complete standard workflow on a real airfoil. By default, it reads the coordinate file:

```
tests/data/NACA2312.dat
```

A different coordinate file can be passed as the first command-line argument:

```
./bin/example path/to/airfoil.dat
```

The input routine accepts empty lines, comment lines beginning with `#`, and data lines containing at least two numerical columns:

```
x y
```

An optional third column, for example a thickness value, is ignored by the example input routine.

7.1 Operations Performed by the Example

The example performs the following steps:

1. reads the airfoil coordinates into two `std::vector<double>` objects,
2. creates an `AirfoilData` object,
3. assigns an airfoil name and both coordinate vectors with `setAirfoilPoints`,
4. checks the contour with `airfoilOK`,
5. calculates the complete airfoil properties,
6. prints selected geometric and inviscid quantities,
7. demonstrates Akima interpolation on the calculated camber line,
8. updates the normalized vortex-sheet and scaled source–sink distributions,

9. redistributes the airfoil to 333 support points,
10. writes four data files for plotting with gnuplot.

The complete property calculation is initiated with:

```
if (!ad.calculateAirfoilProperties())
{
    std::cerr << "calculateAirfoilProperties() failed\n";
    return 1;
}
```

The example then redistributes the contour with:

```
if (ad.redistributeAirfoil(333, RedistributeMode::AllData))
{
    std::cout << ad.getAirfoilX().size()
    << " points active\n";
}
```

The mode `RedistributeMode::AllData` replaces the current contour and recalculates:

- the derived geometry,
- the normalized vortex-sheet distribution,
- the scaled source–sink distribution,
- upper- and lower-surface data,
- first and second chordwise derivatives.

7.2 Example Console Output

For the supplied NACA 2312 coordinate file, version 1.0.0 produces the following output:

```
AirfoilData 1.0.0 (1.0.0)

Airfoil: tests/data/NACA2312.dat  (255 points)

Geometry:
Thickness               = 0.1200
Thickness location      = 0.2998
Camber                  = 0.0211
Camber location         = 0.2961
TE thickness            = 0.0025
Area                    = 0.0819
Nose thickness at 1.25% = 0.0199
alpha0 (inviscid)       = -1.9624
cm0    (inviscid)       = -0.0454

Camber line (Akima):  x=0.2500 y=0.0205  x=0.5000 y=0.0191  x=0.7500 y=0.0120
Circulation:     121 support points
Source strength: 128 support points

Redistribution to 333 points -> 333 points active
Output files written: example_airfoil.dat, example_camber.dat,
example_circ_source.dat, example_grad_curv.dat
```

The displayed values are rounded to four decimal places. The numerical data stored in the output files use scientific notation with eight digits after the decimal point.

For a normalized airfoil, thickness, camber, area, and coordinate values are expressed relative to a chord length of one. The value returned by `getAlpha0Geo` is expressed in degrees.

In the console output, the short labels *Circulation* and *Source strength* refer respectively to the normalized vortex-sheet strength k/U_∞ and the scaled source distribution $q/(2U_\infty) = dz^{(t)}/dx$.

7.3 Generated Data Files

The example writes four whitespace-separated data files to the current working directory. Lines beginning with `#` contain column descriptions.

7.3.1 `example_airfoil.dat`

This file contains the redistributed airfoil contour and the signed thickness distribution.

Column	Description
1	airfoil x coordinate
2	airfoil y coordinate
3	thickness-distribution x coordinate
4	signed half-thickness coordinate

The thickness distribution follows the complete contour order. Its lower branch has a negative sign and its upper branch has a positive sign. It is therefore suitable for displaying the thickness distribution as a symmetric contour around the zero line.

7.3.2 `example_camber.dat`

This file contains the calculated camber line.

Column	Description
1	camber-line x coordinate
2	camber-line y coordinate

The camber-line coordinates run from the leading edge towards the trailing edge.

7.3.3 `example_circ_source.dat`

The file contains two independent gnuplot data blocks:

Data block	Contents
`index 0`	Chordwise coordinate and normalized vortex-sheet strength k/U_∞ at the reference lift coefficient $c_l = 0.5$.
`index 1`	Chordwise coordinate and scaled source–sink distribution $q/(2U_\infty) = dz^{(t)}/dx$.

The individual blocks can be selected in gnuplot with:

```
"example_circ_source.dat" index 0 using 1:2
"example_circ_source.dat" index 1 using 1:2
```

7.3.4 `example_grad_curv.dat`

This file contains derivative data for the upper and lower surfaces.

Column	Description
1	upper-surface x coordinate
2	upper-surface first derivative dy/dx
3	upper-surface second derivative d^2y/dx^2
4	lower-surface x coordinate
5	lower-surface first derivative dy/dx
6	lower-surface second derivative d^2y/dx^2

Note

The public API uses the term *curvature* for the values returned by `getCurvatureDataUpper` and `getCurvatureDataLower`. In the current implementation, these values are the second chordwise derivatives d^2y/dx^2. They are not transformed into the differential-geometric curvature

$$\kappa = \frac{d^2y/dx^2}{(1 + (dy/dx)^2)^{3/2}}.$$

8 Plot Generation with Gnuplot

The supplied script:

```
plot/plot.gnu
```

reads the four data files written by the example program and produces four PNG images in the `plot` directory.

The complete process can be started with:

```
make plot
```

This target builds the example, runs it with the default airfoil, and calls:

```
gnuplot plot/plot.gnu
```

The PNG terminal uses a resolution of 2700×1560 pixels and is intended to provide sufficient resolution for printed documentation.

If gnuplot is not available, the example is still executed and the four data files are written. The Makefile then prints a message indicating that only the data files were generated.

8.1 Airfoil and Thickness Distribution

The first plot is written to:

```
plot/example_airfoil.png
```

It compares the redistributed airfoil contour with the signed thickness distribution.

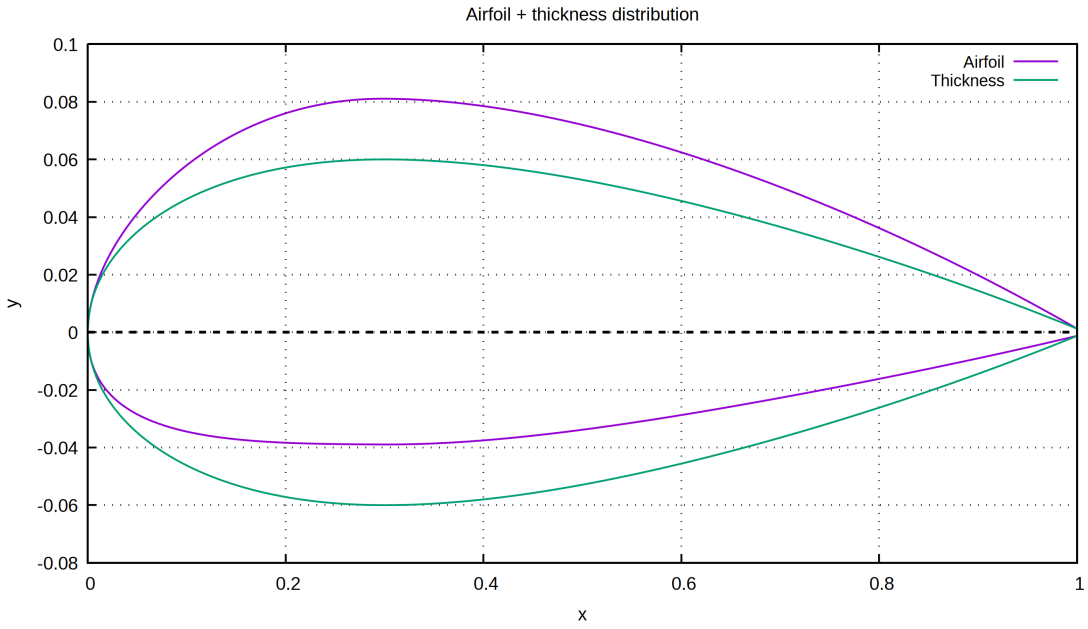


Figure 1: Redistributed NACA 2312 contour and signed thickness distribution. The thickness distribution is displayed around the zero line and shows the chordwise variation of the airfoil thickness.

8.2 Camber Line

The second plot is written to:

```
plot/example_camber.png
```

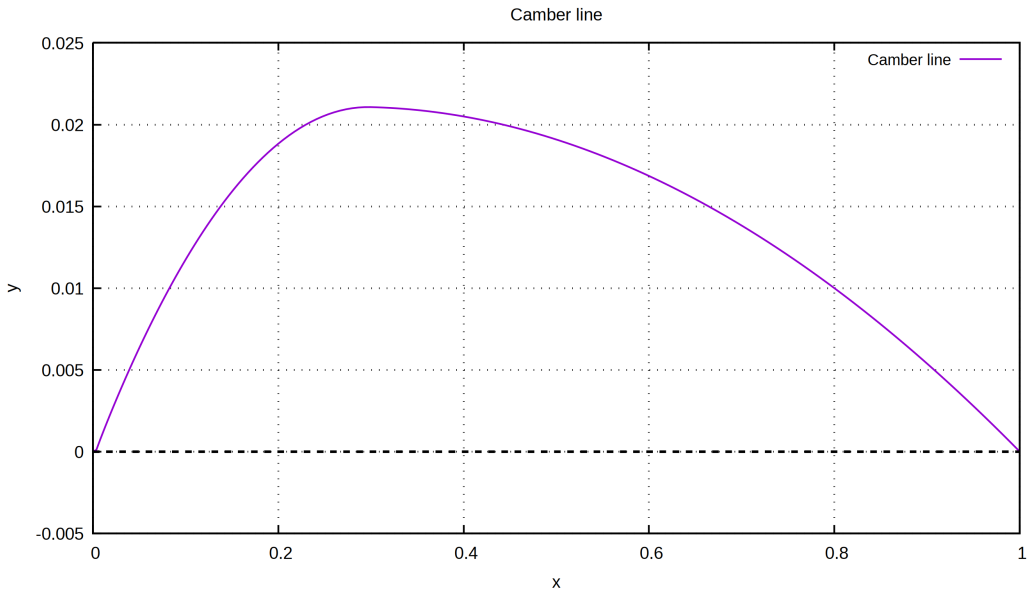


Figure 2: Camber line calculated from the redistributed NACA 2312 contour.

8.3 Circulation and Source Strength

The third plot is written to:

```
plot/example_circ_source.png
```

The normalized circulation distribution and the scaled source–sink distribution are displayed on a common vertical axis. The negative part of the scaled source distribution represents sink regions, where the half-thickness decreases towards the trailing edge.

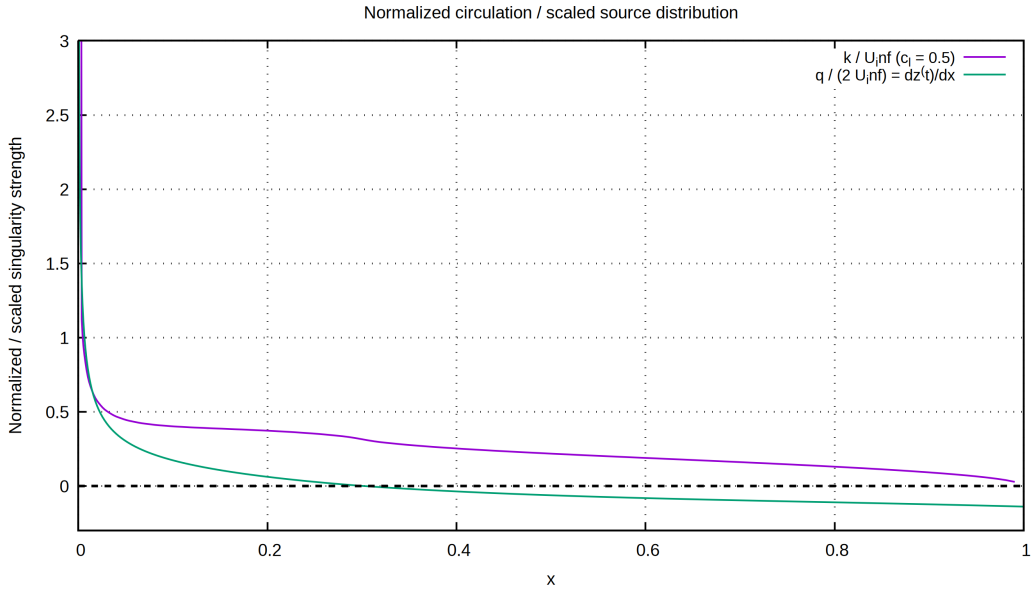


Figure 3: Normalized vortex-sheet distribution k/U_∞ at the reference lift coefficient $c_l = 0.5$ and scaled source–sink distribution $q/(2U_\infty) = dz^{(t)}/dx$. Both distributions are used primarily as geometry descriptors for subsequent airfoil-generation methods.

Note

The gnuplot script uses the fixed vertical range $-0.3 \leq y \leq 3$ for this figure. The circulation peak near the leading edge can exceed the upper plotting limit and is intentionally clipped to make the remaining distributions visible. The numerical values in the data file are not clipped.

8.4 Surface Derivatives

The fourth plot is written to:

```
plot/example_grad_curv.png
```

The first derivatives are displayed on the left vertical axis, while the second derivatives are displayed on the right vertical axis.

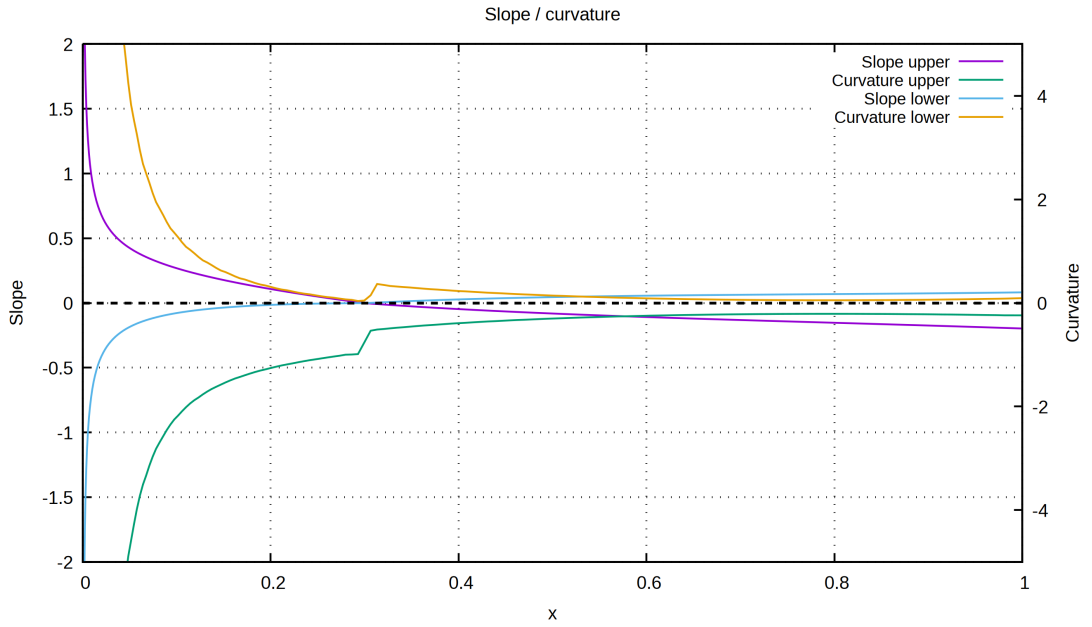


Figure 4: First and second chordwise derivatives of the upper and lower surfaces. The API refers to the second derivatives as curvature data.

Note

For readability, the plot uses the fixed ranges $-2 \leq dy/dx \leq 2$ and $-5 \leq d^2y/dx^2 \leq 5$. Larger derivative values, particularly near the leading edge, are therefore clipped in the figure. The values stored in `example_grad_curv.dat` are not clipped.

9 Building and Testing

The root Makefile builds the example program, invokes the test harness, and generates the example plots.

The expected project layout is:

```
CHANGELOG.md
CITATION.cff
CONTRIBUTING.md
GPLv3
LICENCE
Makefile
README.md
example.cpp
src/
    AirfoilData.h
    AirfoilData.cpp
    AkimaInterpolation.h
    AkimaInterpolation.cpp
tests/
plot/
     plot.gnu
```

The build uses or creates the following directories:

```
build/    reusable core object files
bin/      example executable
lib/      optional static archive
plot/     gnuplot script and generated PNG files
```

9.1 Manual Compilation of the Core Object Files

The library does not require the creation of a static or shared library. Instead, the two implementation files can be compiled into ordinary object files and linked directly into an application.

Compile the core object files with GCC:

```
g++ -std=c++17 -O2 -Wall -Wextra -Isrc \
-c src/AirfoilData.cpp \
-o AirfoilData.o

g++ -std=c++17 -O2 -Wall -Wextra -Isrc \
-c src/AkimaInterpolation.cpp \
-o AkimaInterpolation.o
```

A user program named `main.cpp` can then be compiled and linked with:

```
g++ -std=c++17 -O2 -Wall -Wextra -Isrc \
main.cpp \
AirfoilData.o \
AkimaInterpolation.o \
-o main
```

On Linux and macOS, run the resulting application with:

```
./main
```

On Windows with MSYS2, the executable is normally named:

```
main.exe
```

Applications using only the standalone interpolation class can be linked with:

```
g++ -std=c++17 -O2 -Wall -Wextra -Isrc \
interpolation_example.cpp \
AkimaInterpolation.o \
-o interpolation_example
```

Note

C++ object files are compiler-, platform-, architecture-, and configuration-dependent. Compile the object files with a toolchain and compiler options compatible with those used for the calling application.
In particular, object files produced by different C++ ABI environments should not be assumed to be interchangeable. For reproducible builds, it is recommended to compile the two object files locally from the source release.

9.2 Building the Example

Build the example with:

```
make
```

or explicitly with:

```
make example
```

The executable is written to:

```
bin/example
```

The Makefile first compiles the two core object files and then links the example against them. The corresponding link command is equivalent to:

```
g++ -std=c++17 -Wall -Wextra -Isrc \
example.cpp \
build/AirfoilData.o \
build/AkimaInterpolation.o \
-o bin/example
```

The two implementation files may alternatively be compiled directly together with the example, as shown in the Quick Start section.

9.3 Building the Core Object Files with Make

The two reusable core object files can be built without creating the example executable:

```
make objects
```

The generated files are:

```
build/AirfoilData.o
build/AkimaInterpolation.o
```

A user program can be linked directly against these files:

```
g++ -std=c++17 -O2 -Wall -Wextra -Isrc \
main.cpp \
build/AirfoilData.o \
build/AkimaInterpolation.o \
-o main
```

The ordinary `make example` target uses the same direct-object linking method.

9.4 Optional Static Archive

On systems providing a compatible `ar` tool, including typical Linux, macOS, and MSYS2/MinGW environments, create the optional static archive with:

```
make lib
```

The archive is written to:

```
lib/libAirfoilData.a
```

A user program can then be linked with:

```
g++ -std=c++17 -O2 -Wall -Wextra -Isrc \
main.cpp \
-Llib -lAirfoilData \
-o main
```

The library option must appear after the source or object files that refer to its symbols when using linkers that process libraries from left to right.

9.5 Running the Example

Run the example with its default input file:

```
./bin/example
```

A different coordinate file can be supplied as the first argument:

```
./bin/example path/to/airfoil.dat
```

On Windows with MSYS2, run:

```
./bin/example.exe
```

or

```
./bin/example.exe path/to/airfoil.dat
```

9.6 Running the Standard Test Set

The root target:

```
make test
```

requests the standard developer check set through:

```
make -C tests check
```

The check set uses GCC and Clang, together with clang-tidy, GCC coverage tools, sanitizer support, and the libstdc++ debug mode where available.

A successful run ends with:

```
TEST OVERALL: PASSED
```

This message means that all checks actually executed on the current system passed. It does not necessarily mean that every optional sanitizer or analysis target was executed.

If a required optional compiler runtime or tool is unavailable, the corresponding target is reported as `NOT RUN`. The remaining available checks continue to execute.

Note

Building and using the library does not require the complete developer tool set. The optional tools are needed only for the corresponding test targets.
Contributors and release maintainers should inspect all `NOT RUN` warnings and record which checks were skipped.

9.7 Skipped Optional Checks

Some developer checks require compiler runtime components that are not available in every installation. In particular, an MSYS2 UCRT64 installation may provide GCC or Clang without the sanitizer runtimes required by the corresponding targets.

The test Makefile detects these cases. Instead of failing the complete check, it skips the unavailable optional target and emits a warning, for example:

```
WARNING: san-gcc NOT RUN - the address/undefined sanitizer runtime
(libasan/libubsan) is not installed for g++.

WARNING: san-clang NOT RUN - the address/undefined sanitizer runtime
(compiler-rt) is not installed for clang++.
```

A skipped optional check is not treated as a failed numerical regression. The remaining available checks continue to run.

Note

A skipped sanitizer check provides no sanitizer result. It must not be described as a successful sanitizer run.
Before a release, the sanitizer targets should be executed successfully on at least one supported environment that provides the required runtime libraries.

Example MSYS2 UCRT64 Result

A test run without installed sanitizer runtimes may produce output similar to:

```
PASS: 993 records match the golden reference
(abs tol 1e-12, rel tol 1e-9).
PASS: 993 records match the golden reference
(abs tol 1e-12, rel tol 1e-9).
WARNING: san-gcc NOT RUN - the address/undefined sanitizer runtime
(libasan/libubsan) is not installed for g++.
WARNING: san-clang NOT RUN - the address/undefined sanitizer runtime
(compiler-rt) is not installed for clang++.
WARNING: san-full-gcc NOT RUN - the address/undefined sanitizer runtime
(libasan/libubsan) is not installed for g++.
PASS: 993 records match the golden reference
(abs tol 1e-12, rel tol 1e-9).
PASS: 993 records match the golden reference
(abs tol 1e-12, rel tol 1e-9).
Method coverage: 161/161 methods tested
TEST OVERALL: PASSED
```

This result indicates that:

- all executed regression comparisons passed,
- the three sanitizer targets were skipped,
- the remaining requested checks completed successfully,
- all 161 methods recognized by the coverage-reporting filter were executed at least once.

Method coverage reports execution. It does not establish complete branch, path, condition, or numerical-domain coverage.

9.8 Test-Harness Design

The regression harness consists of a common scenario corpus and two small driver programs:

- `golden_generator.cpp` writes the scenario results to the golden reference file,
- `test_airfoildata.cpp` Runs the same scenarios and compares the generated records with the existing golden reference. Structural and textual fields are compared exactly. Serialized finite floating-point values are interpreted numerically and compared using the configured absolute and relative tolerances,
- `harness_common.cpp` contains the scenarios and deterministic serialization functions.

This design ensures that the golden generator and the regression test execute the same sequence of library calls. They differ only in how the emitted results are processed.

Floating-point values are serialized with the C `%a` format. This represents the stored binary

floating-point value as a hexadecimal floating-point number and avoids loss of information through decimal formatting.

The principal regression profile is:

```
tests/data/NACA4618.dat
```

The example program and the regression harness intentionally use different airfoils:

- NACA 2312 for the documented example,
- NACA 4618 for the principal regression scenarios.

The scenario corpus exercises every public method of `AirfoilData` and `AkimaInterpolation` at least once. It also includes invalid and boundary inputs such as:

- empty vectors and single-point input,
- inconsistent vector lengths,
- too few support points,
- duplicated or non-monotone chordwise coordinates,
- large point sets,
- NaN and infinity,
- invalid redistribution point counts,
- uninitialized Akima interpolation,
- non-ascending Akima support points,
- finite data producing non-finite Akima slopes,
- output callback and console fallback behavior,
- output suppression and diagnostic verbosity.

Private helper methods are tested indirectly through the public methods that use them.

9.9 Individual Test Targets

Target	Description
`test-gcc`	Builds with GCC and compares the scenario output with the golden reference.
`test-clang`	Builds with Clang and compares the output with the same reference.
`san-gcc`	Runs AddressSanitizer and UndefinedBehaviorSanitizer with GCC. The target is reported as skipped if the required runtime libraries are unavailable.
`san-clang`	Runs the Clang sanitizer build, including implicit-conversion checks. The target is reported as skipped if compiler-rt sanitizer support is unavailable.
`san-full-gcc`	Runs the expanded GCC undefined-behavior checks. The target is reported as skipped if the GCC sanitizer runtimes are unavailable.
`dbg-gcc`	Uses the debug mode of libstdc++ for container and iterator checking.
`coverage`	Produces gcov data and reports method coverage for the core classes.
`analyzer`	Runs the GCC static analyzer on the two core implementation files.
`tidy`	Runs the clang-analyzer checks through clang-tidy.
`memcheck`	Runs the GCC test executable with Valgrind Memcheck.
`sharp`	Runs the extended sanitizer, debug, coverage, analyzer, clang-tidy, and Valgrind checks.
`strict`	Builds the GCC regression test with warnings treated as errors.

Examples:

```
make -C tests test-gcc
make -C tests san-gcc
make -C tests coverage
make -C tests memcheck
make -C tests sharp
```

9.10 Updating the Golden Reference

The committed file:

```
tests/data/golden.txt
```

defines the expected record structure, textual output, and numerical reference values of the documented library version.

Tags, dimensions, integer and Boolean values, diagnostic text, record order, and the record count are compared exactly. Finite floating-point values are compared according to:

$$|g - a| \leq 10^{-12} + 10^{-9} \max\left(|g|, |a|\right).$$

A floating-point value that differs from its golden value but remains inside this tolerance does not require a golden-reference update.

A test failure can indicate:

- a structural or textual difference,
- a missing or additional record,
- a vector-size or status difference,
- a floating-point deviation exceeding the accepted tolerance,
- an unexpected change between finite and non-finite values.

Do not regenerate the golden reference merely to make a failing test pass. First determine whether a difference is:

1. an unintended regression,
2. an intended numerical correction,
3. an intended API or diagnostic-message change,
4. a platform-specific difference requiring further investigation.

Only intended and reviewed changes should be incorporated into the reference.

Generate a new reference intentionally with:

```
make -C tests golden-gcc
```

Inspect the complete difference:

```
git diff -- tests/data/golden.txt
```

After reviewing the new reference, rerun at least:

```
make -C tests test-gcc
make -C tests test-clang
make -C tests san-gcc
make -C tests san-clang
```

Note

The normal regression target must compare against the existing committed golden reference. It must not regenerate the reference automatically before comparison.

10 Akima Interpolation

The class `AkimaInterpolation` implements one-dimensional piecewise cubic Akima interpolation. It is independent of `AirfoilData` and can be used for other numerical data.

10.1 Input Requirements

The support points supplied to `init` must satisfy all of the following conditions:

- the x and y vectors have the same length,
- at least two support points are present,
- all x and y values are finite,
- the x coordinates are strictly ascending,
- the calculated support-point slopes remain finite.

Nearly identical but strictly ascending x coordinates are accepted. However, extremely small coordinate differences can produce very large secant slopes. Initialization fails if the calculated slopes are not finite.

A failed call to `init` invalidates the interpolation object. A previously initialized state is not retained.

10.2 Basic Example

```
1  #include "AkimaInterpolation.h"
2
3  #include <iostream>
4  #include <vector>
5
6  int main()
7  {
8      std::vector<double> x =
9      {0.0, 0.25, 0.5, 0.75, 1.0};
10
11     std::vector<double> y =
12     {0.0, 0.05, 0.04, 0.02, 0.0};
13
14     AkimaInterpolation interpolation;
15
```

```
    if (!interpolation.init(x, y))
    {
        std::cerr << "Akima initialization failed.\n";
        return 1;
    }

    double value = 0.0;
    double firstDerivative = 0.0;
    double secondDerivative = 0.0;

    if (!interpolation.interpolate(
    0.4,
    value,
    firstDerivative,
    secondDerivative))
    {
        std::cerr << "Akima interpolation failed.\n";
        return 1;
    }

    std::cout << "y       = " << value << "\n";
    std::cout << "dy/dx   = " << firstDerivative << "\n";
    std::cout << "d2y/dx2 = " << secondDerivative << "\n";

    return 0;
}
```

10.3 Available Results

Three overloads of `interpolate` are available:

Signature	Result
`interpolate(x,value)`	Calculates the interpolated value.
`interpolate(x,value,firstDerivative)`	Calculates the value and first derivative.
`interpolate(x,value,firstDerivative,secondDerivative)`	Calculates the value, first derivative, and second derivative.

The requested abscissa must be finite. The method returns `false` if the object has not been initialized successfully or if the requested abscissa is not finite.

Output parameters should be used only after a successful call.

Note

Requests outside the support-point interval are accepted by the current implementation. Applications that require a specific extrapolation model should nevertheless check the requested abscissa explicitly and restrict it to the support-point interval if appropriate.

10.4 Recommended Akima Workflow

An `AkimaInterpolation` object is normally initialized once and then evaluated at several target abscissas.

The recommended workflow is:

1. prepare strictly ascending support-point coordinates,
2. construct an `AkimaInterpolation` object,
3. initialize the object with `init`,
4. check the returned status,
5. call the required `interpolate` overload for every target abscissa,
6. use the output values only after a successful interpolation call.

```
AkimaInterpolation interpolation;

if (!interpolation.init(x, y))
{
    std::cerr << "Akima initialization failed.\n";
    return 1;
}

for (double targetX : targetCoordinates)
{
    double targetY = 0.0;

    if (!interpolation.interpolate(targetX, targetY))
    {
        std::cerr << "Akima interpolation failed.\n";
        return 1;
    }

    std::cout << targetX << " "
    << targetY << "\n";
}
```

The support-point vectors are copied into the interpolation object during initialization. They may therefore be modified or destroyed after `init` has returned successfully without affecting the initialized interpolator.

10.5 Interpolating an Airfoil Camber Line

A typical application of `AkimaInterpolation` is the evaluation of a camber line calculated by `AirfoilData`.

After a successful call to `calculateAirfoilProperties`, the camber-line coordinates are available through `getCamberLineX` and `getCamberLineY`. The chordwise camber-line coordinates are stored on an ascending grid and can therefore be passed directly to `AkimaInterpolation::init`.

```
#include "AirfoilData.h"
#include "AkimaInterpolation.h"

#include <iostream>
#include <vector>

```

```
// ...

AirfoilData airfoil;

if (!airfoil.setAirfoilPoints(x, y))
{
    std::cerr
    << "Airfoil coordinate vectors are inconsistent.\n";
    return 1;
}

if (!airfoil.calculateAirfoilProperties())
{
    std::cerr
    << "Airfoil property calculation failed.\n";
    return 1;
}

const std::vector<double>& camberX =
airfoil.getCamberLineX();

const std::vector<double>& camberY =
airfoil.getCamberLineY();

AkimaInterpolation camberInterpolation;

if (!camberInterpolation.init(camberX, camberY))
{
    std::cerr
    << "Camber-line interpolation initialization failed.\n";
    return 1;
}

const double targetX = 0.5;
double targetY = 0.0;
double firstDerivative = 0.0;
double secondDerivative = 0.0;

if (!camberInterpolation.interpolate(
targetX,
targetY,
firstDerivative,
secondDerivative))
{
    std::cerr
    << "Camber-line interpolation failed.\n";
    return 1;
}

std::cout << "At x/c = " << targetX << ":\n"
<< "  camber       = " << targetY << "\n"
<< "  slope        = " << firstDerivative << "\n"
<< "  second deriv = " << secondDerivative << "\n";
```

The three returned quantities are:

$$y(x), \qquad \frac{dy}{dx}, \qquad \frac{d^2y}{dx^2}.$$

For the example NACA 2312 airfoil, value-only interpolation at three chordwise positions produces:

```
Camber line (Akima):  x=0.2500 y=0.0205  x=0.5000 y=0.0191  x=0.7500 y=0.0120
```

Note

The camber-line vectors must contain at least two points and the chordwise coordinates must be strictly ascending. Always check both the property-calculation result and the Akima initialization result before evaluating the interpolation.

10.6 AkimaInterpolation API Reference

class AkimaInterpolation

Header

`AkimaInterpolation.h`

Description

`AkimaInterpolation` implements one-dimensional piecewise cubic Akima interpolation. An object stores copies of the supplied support-point coordinates and the calculated slopes at those points. After successful initialization, the object can be evaluated repeatedly without recalculating the support slopes.
The class provides:

- interpolation of the function value,
- interpolation of the value and first derivative,
- interpolation of the value and first and second derivatives.

The class is independent of `AirfoilData` and can also be used for other numerical data.

Notes

`AkimaInterpolation` follows the Rule of Zero. It contains standard value and vector members and does not require a user-defined destructor, copy constructor, or assignment operator.
Copying an initialized object copies its complete interpolation state.
A default-constructed object is not initialized. Calling `interpolate` before successful initialization returns `false`.

Example

```
AkimaInterpolation interpolation;

if (!interpolation.init(x, y))
{
    return 1;
}

double value = 0.0;
```

class AkimaInterpolation *(continued)*

```
if (!interpolation.interpolate(0.5, value))
{
    return 1;
}
```

AkimaInterpolation::AkimaInterpolation

Signature

```
AkimaInterpolation();
```

Description

Constructs an empty, uninitialized interpolation object.

Notes

The object must be initialized with `init` before an interpolation method can succeed.

AkimaInterpolation::init

Signature

```
bool init(
        const std::vector<double>& x,
        const std::vector<double>& y);
```

Description

Stores the support points and calculates the Akima slopes required for subsequent interpolation.
The support-point vectors are copied into the object.

Parameters

`x`	Strictly ascending support-point abscissas. At least two values are required. All values must be finite.
`y`	Support-point ordinates. The vector must have the same number of elements as `x`. All values must be finite.

Returns

`true` if the support points were accepted and all required slopes were calculated successfully. Returns `false` if:

- fewer than two support points are supplied,
- the vector lengths differ,
- an x or y value is non-finite,
- the x values are duplicated or not strictly ascending,
- a calculated support-point slope is non-finite.

Notes

A failed initialization invalidates the complete interpolation object. A previously valid

AkimaInterpolation::init *(continued)*

interpolation state is discarded.
Nearly identical x values are accepted if they remain strictly ascending. Such support points can nevertheless produce very large secant slopes. Initialization fails if the resulting stored slopes are not finite.
For exactly two support points, the interpolator uses the common secant slope at both endpoints and therefore represents a linear function between the two points.
The method is marked with `[[nodiscard]]`; its return value should always be checked.

Example

```
std::vector<double> x =
{0.0, 0.25, 0.5, 0.75, 1.0};

std::vector<double> y =
{0.0, 0.05, 0.04, 0.02, 0.0};

AkimaInterpolation interpolation;

if (!interpolation.init(x, y))
{
    std::cerr
    << "Invalid interpolation support points.\n";
    return 1;
}
```

AkimaInterpolation::interpolate — value

Signature

```
bool interpolate(
        double desiredAbscissa,
        double& value) const;
```

Description

Calculates the interpolated function value at the requested abscissa.

Parameters

`desiredAbscissa`	Target abscissa. The value must be finite.
`value`	Output parameter receiving the interpolated function value.

Returns

`true` if the interpolation was performed.
Returns `false` if the object has not been initialized successfully or if `desiredAbscissa` is NaN or positive or negative infinity.

Notes

The output parameter remains unchanged when the method rejects the call before interpolation.
Requests outside the support-point interval are accepted by the current implementation. Applications requiring interpolation only should check the requested abscissa against the support-point range before calling the method.
The method is marked with `[[nodiscard]]`.

AkimaInterpolation::interpolate — value *(continued)*

Example

```
double value = 0.0;

if (!interpolation.interpolate(0.4, value))
{
    std::cerr << "Interpolation failed.\n";
}
```

AkimaInterpolation::interpolate — first derivative

Signature

```
bool interpolate(
        double desiredAbscissa,
        double& value,
        double& firstDerivative) const;
```

Description

Calculates the interpolated function value and its first derivative at the requested abscissa.

Parameters

`desiredAbscissa`	Target abscissa. The value must be finite.
`value`	Output parameter receiving the interpolated function value.
`firstDerivative`	Output parameter receiving dy/dx.

Returns

`true` if the interpolation and derivative calculation were performed.
Returns `false` if the object is not initialized or the target abscissa is non-finite.

Notes

The output parameters should be used only after a successful call.

Example

```
double value = 0.0;
double slope = 0.0;

if (interpolation.interpolate(
0.4,
value,
slope))
{
    std::cout << "y = " << value << "\n";
    std::cout << "dy/dx = " << slope << "\n";
}
```

AkimaInterpolation::interpolate — second derivative

Signature

```
bool interpolate(
        double desiredAbscissa,
        double& value,
        double& firstDerivative,
        double& secondDerivative) const;
```

Description

Calculates the interpolated function value together with its first and second derivatives.

Parameters

`desiredAbscissa`	Target abscissa. The value must be finite.
`value`	Output parameter receiving the interpolated function value.
`firstDerivative`	Output parameter receiving dy/dx.
`secondDerivative`	Output parameter receiving d^2y/dx^2.

Returns

`true` if the value and both derivatives were calculated.
Returns `false` if the object is not initialized or the target abscissa is non-finite.

Notes

At an interior support point, the first derivative is the Akima slope stored for that support point.
The piecewise cubic interpolation is continuously differentiable in its first derivative. The second derivative can differ on the two sides of an interior support point. The implementation uses its defined support-point treatment when the second derivative is requested exactly at such a point.
The output parameters should be used only after a successful call.

Example

```
double value = 0.0;
double first = 0.0;
double second = 0.0;

if (interpolation.interpolate(
0.4,
value,
first,
second))
{
    std::cout << "y       = " << value << "\n";
    std::cout << "dy/dx   = " << first << "\n";
    std::cout << "d2y/dx2 = " << second << "\n";
}
```

10.7 Writing an Interpolated Data Table

The following example evaluates an initialized interpolator at 101 evenly spaced chordwise positions and writes the value and two derivatives to a file.

```cpp
#include "AkimaInterpolation.h"

#include <fstream>
#include <iostream>
#include <vector>

// x and y contain the support points.

AkimaInterpolation interpolation;

if (!interpolation.init(x, y))
{
    std::cerr << "Akima initialization failed.\n";
    return 1;
}

std::ofstream output("interpolated_data.dat");

if (!output)
{
    std::cerr << "Could not open output file.\n";
    return 1;
}

output << "# x y dy_dx d2y_dx2\n";

for (std::size_t i = 0; i <= 100; ++i)
{
    const double targetX =
    static_cast<double>(i) / 100.0;

    double targetY = 0.0;
    double firstDerivative = 0.0;
    double secondDerivative = 0.0;

    if (!interpolation.interpolate(
    targetX,
    targetY,
    firstDerivative,
    secondDerivative))
    {
        std::cerr
        << "Interpolation failed at x = "
        << targetX << "\n";
        return 1;
    }

    output << targetX << " "
    << targetY << " "
    << firstDerivative << " "
    << secondDerivative << "\n";
}
```

The generated columns can be plotted with gnuplot, for example:

```
plot "interpolated_data.dat" using 1:2 with lines title "value"
```

10.8 Recommendations for Akima Interpolation

- Initialize an `AkimaInterpolation` object once and reuse it for several target abscissas.
- Always check the return values of `init` and `interpolate`.
- Supply strictly ascending and finite support-point abscissas.
- Do not pass the unmodified x coordinates of a complete airfoil contour directly to `AkimaInterpolation`; the usual complete contour is not monotone.
- Select the simplest `interpolate` overload that provides the required results.
- Check the target range explicitly when extrapolation is not intended.
- Verify interpolated values with `std::isfinite` when extreme support-point values or long extrapolation distances are possible.

10.9 Akima Interpolation Troubleshooting

`init` Returns `false`

Check the following conditions:

- both support-point vectors contain the same number of elements,
- at least two points are present,
- all support-point values are finite,
- the x values are strictly ascending,
- no duplicated x values are present,
- adjacent x values are not so close that the calculated slopes overflow.

A common error with complete airfoil contours is passing the original chordwise coordinates directly to `AkimaInterpolation`. A complete airfoil contour normally follows:

$$1 \rightarrow 0 \rightarrow 1$$

and is therefore not strictly ascending. It must first be split into monotone sides or transformed in the same manner as the corresponding `AirfoilData` methods.

`interpolate` Returns `false`

Check that:

- `init` previously returned `true`,
- no later failed call to `init` invalidated the object,
- the requested abscissa is finite.

Unexpected Results Outside the Support Range

The current implementation accepts finite target abscissas outside the support-point interval. If extrapolation is not intended, check the range in the calling application:

```
if (targetX < x.front() || targetX > x.back())
{
    std::cerr << "Target outside support range.\n";
    return 1;
}
```

Non-Finite Results for Extreme Queries

Initialization verifies the finiteness of the support points and calculated support slopes. Nevertheless, evaluating a cubic polynomial at an extremely large but finite target abscissa can exceed the representable floating-point range. Applications using extrapolation over large distances should additionally verify the returned numerical values with `std::isfinite`.

11 Public Interface Overview

The public interface of `AirfoilData` is intentionally extensive. It supports both ordinary airfoil-processing workflows and specialized engineering applications.

For easier orientation, this manual divides the public interface into three groups.

Basic API
: Methods required for assigning an airfoil contour, validating it, calculating the standard properties, redistributing the contour, saving coordinates, and retrieving the principal results.

Advanced API
: Methods that expose individual processing steps, including normalization, scaling, rotation, interpolation support-point mapping, derivative calculation, camber and thickness extraction, circulation, source strength, and repair of chordwise contour reversals.

Integration API
: Methods intended mainly for integration into a larger application. These include the output callback, diagnostic control, storage of external aerodynamic data, pressure-distribution data, conformal-mapping data, and direct setters for intermediate result vectors.

The classification is a documentation aid only. All listed methods are part of the public C++ interface and may be used by an application.

11.1 Recommended Basic Workflow

The recommended Basic API workflow is:

1. construct an `AirfoilData` object,
2. optionally assign a descriptive airfoil name,
3. assign both coordinate vectors with `setAirfoilPoints`,
4. validate the contour with `airfoilOK`,
5. calculate all standard properties with `calculateAirfoilProperties`,
6. retrieve the required scalar values or distributions,
7. optionally redistribute the airfoil,
8. optionally mark the object as created and save its coordinates.

```
1  AirfoilData airfoil;
2
3  airfoil.setAirfoilName("NACA2312");
4
5  if (!airfoil.setAirfoilPoints(x, y))
6  {
7      std::cerr
```

```
    << "Airfoil coordinate vectors are inconsistent.\n";
    return 1;
}

if (!airfoil.airfoilOK(TrailingEdgeCheck::Full))
{
    return 1;
}

if (!airfoil.calculateAirfoilProperties())
{
    return 1;
}

std::cout << airfoil.getThickness() << "\n";
std::cout << airfoil.getCamber() << "\n";
```

The method `setAirfoilPoints` assigns both coordinate vectors in one operation. It is the preferred method for loading a complete airfoil contour. The method rejects coordinate vectors of different lengths and leaves the object unchanged in that case.

The method `airfoilOK` performs geometric contour checks. `calculateAirfoilProperties` then performs the complete standard calculation pipeline.

Note

`setAirfoilPoints` checks whether the two coordinate vectors have equal lengths. It does not by itself establish that the coordinates form a valid airfoil. Use `airfoilOK` before performing the complete property calculation.

Note

Most calculation methods return `bool`. Always check the return value before using output parameters or newly calculated object data. Methods marked with the C++ attribute `[[nodiscard]]` are specifically intended not to be ignored.

12 Public Types

The public enumerations select processing modes without using undocumented integer flags. This makes method calls more explicit and prevents invalid combinations of arbitrary numerical values.

12.1 `NoseDetectionMode`

enum class NoseDetectionMode

Description

Selects the method used to determine the leading-edge index of an airfoil contour.

enum class NoseDetectionMode *(continued)*

Value	Description
`LongestSecant`	Determines the leading-edge candidate as the interior contour point having the greatest Euclidean distance from the midpoint of the two contour endpoints. For a closed trailing edge, this midpoint is the trailing-edge point itself. This avoids a corner bias for profiles with finite trailing-edge thickness.
`LocalMinimum`	Determines the leading-edge candidate from a local minimum of the chordwise coordinate. This mode is used primarily for contours that are already oriented in the conventional trailing-edge–leading-edge–trailing-edge order.

Notes

The two modes serve different purposes. `LongestSecant` establishes a chord reference that remains suitable for translated or rotated profiles and for profiles with an open trailing edge. `LocalMinimum` uses the current coordinate-system orientation.
Detection of a candidate index and validation of a geometrically usable leading edge are separate operations. Methods using the detected index may therefore perform additional checks.

12.2 `ThicknessMode`

enum class ThicknessMode

Description

Selects the definition used by `calculateThickness`.

Value	Description
`Absolute`	Calculates the vertical distance between the upper and lower surfaces at equal chordwise positions. In one-sided mode, the absolute ordinate of the supplied side is used.
`Projected`	Determines the maximum projected extent of the upper and lower sides separately and adds the two values. A meaningful chordwise location of the maximum is not provided in this mode.

Notes

`Projected` requires `SideMode::TwoSided`.

12.3 XDistributionMode

enum class XDistributionMode

Description

Selects the chordwise support-point distribution generated by `createXDistribution`.

Value	Description
`Cosine`	Generates a full cosine distribution in contour order, from the trailing edge to the leading edge and back to the trailing edge. Points are clustered near both the leading and trailing edges.
`HalfCosine`	Generates the default half-cosine contour distribution. Points are clustered mainly near the leading edge, while the spacing near the trailing edge is wider.

Notes

Both distributions describe the complete contour coordinate pattern $x = 1 \to 0 \to 1$, not a one-sided monotonically ascending grid.

12.4 ThicknessJob

enum class ThicknessJob

Description

Selects the operation performed by `applyThickness`.

Value	Description
`ScaleOnly`	Applies the ordinary scaling path without changing the current airfoil thickness. The ordinate scaling coefficient remains one.
`FixedThickness`	Scales all ordinate values so that the calculated maximum thickness equals the requested thickness.

12.5 SideMode

enum class SideMode

Description

Specifies whether a calculation operates on a complete two-sided airfoil contour or on one monotone side.

Value	Description
`TwoSided`	The input contains lower and upper surfaces with an interior leading-edge point.
`OneSided`	The input contains one side on an ascending chordwise support grid.

12.6 CamberDifferenceMode

enum class CamberDifferenceMode

Description

Selects the geometric definition used when extracting the camber and thickness distributions.

Value	Description
`Simple`	Uses the vertical midpoint between the upper and lower surfaces at equal chordwise positions.
`Correct`	Reserved for a definition based on the distance perpendicular to the local camber line.

Notes

In version 1.0.0, `Correct` is not implemented. Passing this value to `extractCamberLine` causes the method to report an error and return `false`.

12.7 SortMode

enum class SortMode

Description

Selects whether `normalizeAirfoil` may correct the contour orientation.

Value	Description
`None`	Keeps the supplied contour order.
`Sort`	Corrects the contour orientation when the supplied order is opposite to the convention expected by the normalization process.

Notes

`Sort` does not perform an arbitrary independent sort of all coordinate points. The correspondence between each x and y value is preserved.

12.8 InterpolationMode

enum class InterpolationMode

Description

Selects whether normalization retains the number of input points or maps the contour onto a supplied target distribution.

enum class InterpolationMode *(continued)*

Value	Description
`None`	Does not map onto a new support-point distribution. The output vectors must have the same size as the input vectors.
`Map`	Uses the chordwise values initially supplied in `xOut` as the target support-point distribution. The number of output points is determined by `xOut.size()`.

12.9 `RedistributeMode`

enum class RedistributeMode

Description

Selects the scope of the update performed by `redistributeAirfoil`.

Value	Description
`AllData`	Replaces the contour and recalculates the complete set of derived airfoil data.
`AirfoilOnly`	Replaces only the airfoil contour and clears derived geometry and aerodynamic distributions.

12.10 `RecalcMode`

enum class RecalcMode

Description

Controls whether updates of the normalized vortex-sheet or scaled source–sink distribution first recompute the camber and thickness distributions.

Value	Description
`Recompute`	Recalculates the camber and thickness distributions before the requested update.
`Keep`	Uses the distributions already stored in the object. The operation fails if the required data are not available or inconsistent.

12.11 `TrailingEdgeCheck`

enum class TrailingEdgeCheck

Description

Selects the chordwise range checked by `airfoilOK`.

enum class TrailingEdgeCheck *(continued)*

Value	Description
`Full`	Checks the complete chord up to the trailing edge.
`SkipNearEdge`	Omits the region close to the trailing edge. In version 1.0.0, the check is limited to $x <$ `AirfoilDataConstants::kTrailingEdgeLimit`, whose value is 0.95.

12.12 `AeroScalars`

struct AeroScalars

Description

Bundles the scalar aerodynamic values supplied to `setAerodynamicData`.
Grouping the scalar values in one structure keeps the method signature compact and reduces the risk of accidentally exchanging parameters of the same C++ type.

Member	Description
`double alpha0`	Externally calculated zero-lift angle. The unit is determined by the supplying application; degrees are recommended for consistency with the remaining angle-related interface.
`double cl0`	Lift coefficient at zero angle of attack.
`double cd0`	Drag coefficient at the application-defined zero-lift or reference condition.
`double cm0`	Pitching-moment coefficient at the application-defined zero-lift or reference condition.
`double clMax`	Maximum lift coefficient.
`double eMax`	Maximum efficiency or glide-ratio value.

Notes

All members are initialized to zero by the default member initializers.
`setAerodynamicData` requires all six scalar values to be finite.
These externally supplied values are distinct from the internally calculated inviscid geometric values `getAlpha0Geo()` and `getCm0Geo()`.

Example

```
AeroScalars scalars;
scalars.alpha0 = -2.1;
scalars.cl0 = 0.23;
scalars.cd0 = 0.009;
scalars.cm0 = -0.045;
scalars.clMax = 1.35;
scalars.eMax = 42.0;
```

12.13 AirfoilDataMessageType

enum class AirfoilDataMessageType

Description

Classifies diagnostic messages emitted by the library.

Value	Description
`Normal`	Ordinary user-facing output.
`Info`	Informational or diagnostic output.
`Warning`	Recoverable warning.
`Error`	Error preventing the requested operation.
`WarningSevere`	Severe warning requiring particular attention.
`ErrorSevere`	Severe error, often involving invalid numerical data.

Notes

If an output callback is installed, the message type is passed to that callback. Otherwise, normal and informational messages are written to `std::cout`; warnings and errors are written to `std::cerr`.

12.14 GradientData

struct GradientData

Description

Bundles the upper- and lower-surface coordinate, slope, and second-derivative vectors returned by `getGradientData`.

Member	Description
`bool valid`	Indicates whether the corresponding vector dimensions are mutually consistent.
`std::vector<double> upperX`	Chordwise coordinates of the upper-surface derivative data.
`std::vector<double> lowerX`	Chordwise coordinates of the lower-surface derivative data.
`std::vector<double> upperGradient`	First derivative dy/dx of the upper surface.
`std::vector<double> lowerGradient`	First derivative dy/dx of the lower surface.
`std::vector<double> upperCurvature`	Second derivative d^2y/dx^2 of the upper surface.
`std::vector<double> lowerCurvature`	Second derivative d^2y/dx^2 of the lower surface.

Notes

The structure contains copies of the stored vectors. Modifying a `GradientData` object does not modify the originating `AirfoilData` object.
The flag `valid` reports dimensional consistency. Applications requiring actual derivative data should additionally check that the vectors are not empty.

struct GradientData *(continued)*

The fields named `upperCurvature` and `lowerCurvature` contain second chordwise derivatives rather than differential-geometric curvature values.

13 Basic API Reference

This section describes the methods recommended for ordinary airfoil processing. The more specialized methods are documented in Section 14.

13.1 Construction and Reset

AirfoilData::AirfoilData

API level: Basic API

Signature

```
AirfoilData();
```

Description

Constructs an empty `AirfoilData` object.
The default state contains no airfoil coordinates and no calculated distributions. Scalar geometry and aerodynamic values are initialized to zero. The default desired support-point count is 255.

Notes

Construction does not create an airfoil. The coordinate vectors must be assigned or initialized before calculations are performed.

AirfoilData::clearAirfoilData

API level: Basic API

Signature

```
void clearAirfoilData();
```

Description

Clears the airfoil coordinate vectors and all calculated geometry and aerodynamic data. The object's created state is reset to `false`.

Notes

The airfoil name and desired target point count are not reset by this method.

13.2 Airfoil Name and Coordinates

AirfoilData::setAirfoilName

API level: Basic API

Signature

```
void setAirfoilName(const std::string& newName);
```

Description

Assigns a descriptive name to the airfoil.

Parameters

`newName`	Name stored in the object. The name is used, for example, in coordinate-file headers written by `saveAirfoilCoordinates`.

AirfoilData::getAirfoilName

API level: Basic API

Signature

```
const std::string& getAirfoilName() const noexcept;
```

Description

Returns the airfoil name stored in the object.

Returns

A constant reference to the stored name.

AirfoilData::setAirfoilPoints

API level: Basic API

Signature

```
bool setAirfoilPoints(
        const std::vector<double>& xCoordinates,
        const std::vector<double>& yCoordinates);
```

Description

Assigns the complete airfoil contour to the object.
This is the preferred method for supplying airfoil coordinate data. Both coordinate vectors are supplied together, allowing their dimensions to be checked before the stored contour is replaced.

Parameters

`xCoordinates`	Chordwise coordinates of the airfoil contour.
`yCoordinates`	Corresponding vertical coordinates. The vector must have the same number of elements as `xCoordinates`.

Returns

AirfoilData::setAirfoilPoints *(continued)*

`true` if both vectors have equal lengths and were stored successfully.
Returns `false` if the vector lengths differ. In that case, the previously stored airfoil coordinates and derived data remain unchanged.

Notes

The coordinate vectors are copied into the object.
On success, all previously calculated or externally supplied data that depend on the old contour are cleared. This includes:

- scalar geometry results,
- upper- and lower-surface coordinates,
- camber and thickness distributions,
- derivative data,
- normalized vortex-sheet and scaled source–sink distributions,
- conformal-mapping data,
- external aerodynamic coefficient data,
- pressure-coefficient data,
- the created state.

The airfoil name and desired target point count are retained.
This method checks only the dimensional consistency of the coordinate pair. Equal vector lengths do not guarantee that:

- enough contour points are present,
- all coordinates are finite,
- the contour has the expected orientation,
- an interior leading edge can be determined,
- the upper and lower surfaces do not intersect.

Use `airfoilOK` for geometric validation.

Example

```
AirfoilData airfoil;

if (!airfoil.setAirfoilPoints(x, y))
{
    std::cerr << "x/y coordinate sizes differ.\n";
    return 1;
}
```

AirfoilData::getAirfoilX / getAirfoilY

API level: Basic API

Signature

```
const std::vector<double>& getAirfoilX() const noexcept;
const std::vector<double>& getAirfoilY() const noexcept;
```

Description

Return the current airfoil contour coordinates.

AirfoilData::getAirfoilX / getAirfoilY *(continued)*

Returns

Constant references to the internally stored coordinate vectors.

Notes

The returned references remain valid only while the corresponding object data are not replaced, cleared, or destroyed.

13.3 Airfoil Validation

AirfoilData::airfoilOK

API level: Basic API

Signature

```
bool airfoilOK(TrailingEdgeCheck trailingEdgeCheck);
```

Description

Checks whether the current contour represents a usable airfoil without upper/lower surface crossings in the selected chordwise range.
The method determines an interior leading edge, constructs an Akima interpolation over the transformed contour, and compares corresponding upper- and lower-surface ordinates.

Parameters

`trailingEdgeCheck`	Selects whether the complete chord or only the region away from the trailing edge is checked.

Returns

`true` if the contour passes the check. Returns `false` if the contour data are inconsistent, no usable interior leading edge is found, interpolation fails, or a surface crossing is detected.

Notes

A return value of `false` may be accompanied by an error or warning message through the configured output mechanism.
`TrailingEdgeCheck::SkipNearEdge` can be useful when small trailing-edge irregularities should not invalidate the complete contour.

13.4 Complete Property Calculation

AirfoilData::calculateAirfoilProperties

API level: Basic API

Signature

```
bool calculateAirfoilProperties();
```

Description

AirfoilData::calculateAirfoilProperties *(continued)*

Performs the complete standard airfoil-property calculation.
The processing sequence includes:

1. extraction of camber and thickness distributions,
2. calculation of maximum thickness, camber, area, nose height, and trailing-edge thickness,
3. calculation of the inviscid zero-lift angle and moment coefficient,
4. calculation of normalized vortex-sheet and scaled source–sink distributions,
5. separation into upper and lower surfaces,
6. calculation of first and second chordwise derivatives.

Returns

`true` if all required calculations succeeded. Returns `false` if any processing step fails.

Notes

The method uses transactional error handling. If a processing step fails, the previous object state is restored.
The returned scalar values and distributions should be used only after successful completion.

Example

```
if (!airfoil.calculateAirfoilProperties())
{
    std::cerr << "Property calculation failed.\n";
    return 1;
}

std::cout << airfoil.getThickness() << "\n";
std::cout << airfoil.getCamber() << "\n";
```

13.5 Airfoil Redistribution

AirfoilData::redistributeAirfoil

API level: Basic API

Signature

```
bool redistributeAirfoil(
        std::size_t nPointsNew,
        RedistributeMode mode);
```

Description

Redistributes the current airfoil contour to a cosine-based support-point distribution and normalizes the resulting contour.

Parameters

AirfoilData::redistributeAirfoil *(continued)*

`nPointsNew`	Requested number of contour points. The value must lie in the interval 99 to 511. An even value is reduced by one to obtain an odd point count.
`mode`	Selects whether only the contour is replaced or whether all derived data are recalculated.

Returns

`true` if redistribution and the selected update operation succeeded. Returns `false` if the point count is outside the supported range, the existing contour is invalid, interpolation or normalization fails, or recalculation fails.

Notes

An odd point count provides one distinct leading-edge point in the generated contour distribution.
If an even value is supplied, the actual number of points is one less than the requested value. For example:

```
airfoil.redistributeAirfoil(
100,
RedistributeMode::AirfoilOnly
);

// The resulting contour contains 99 points.
```

With `RedistributeMode::AllData`, the operation is transactional: the original object is retained if recalculation fails.
With `RedistributeMode::AirfoilOnly`, calculated geometry and aerodynamic data are cleared after the new contour has been stored.

13.6 Principal Scalar Results

AirfoilData geometry getters

API level: Basic API

Signature

```
double getThickness() const noexcept;
double getThicknessLocation() const noexcept;
double getCamber() const noexcept;
double getCamberLocation() const noexcept;
double getTrailingEdgeThickness() const noexcept;
double getArea() const noexcept;
double getDyNose() const noexcept;
double getAlpha0Geo() const noexcept;
double getCm0Geo() const noexcept;
```

Description

Return the principal scalar airfoil properties stored in the object.

AirfoilData geometry getters *(continued)*

Method	Returned quantity
`getThickness`	Maximum vertical airfoil thickness.
`getThicknessLocation`	Chordwise location of the maximum thickness.
`getCamber`	Signed maximum camber.
`getCamberLocation`	Chordwise location of the maximum absolute camber.
`getTrailingEdgeThickness`	Vertical trailing-edge thickness.
`getArea`	Cross-sectional area enclosed by the airfoil contour.
`getDyNose`	Upper-surface ordinate at the standard chordwise nose location $x/c = 0.0125$.
`getAlpha0Geo`	Inviscid geometric zero-lift angle in degrees, calculated using thin-airfoil theory.
`getCm0Geo`	Inviscid geometric zero-lift pitching-moment coefficient calculated from the camber-line Fourier coefficients. In the employed thin-airfoil formulation, this corresponds to the constant quarter-chord moment.

Notes

These methods return the currently stored values and do not initiate a calculation. On a newly constructed or cleared object, the values are zero.
For normalized coordinates, lengths and locations are relative to a chord length of one. The normalized area is relative to the square of the chord length.

13.7 Calculated Distributions

AirfoilData surface-coordinate getters

API level: Basic API

Signature

```
const std::vector<double>& getUpperSurfaceX() const noexcept;
const std::vector<double>& getUpperSurfaceY() const noexcept;
const std::vector<double>& getLowerSurfaceX() const noexcept;
const std::vector<double>& getLowerSurfaceY() const noexcept;
```

Description

Return the separated upper- and lower-surface coordinate vectors.
These vectors are populated by `calculateUpperLowerCoordinates`, which is included in the complete `calculateAirfoilProperties` workflow.

Notes

The upper-surface and lower-surface vectors may have different lengths. Each x vector corresponds element by element to its associated y vector.

AirfoilData camber-line getters

API level: Basic API

Signature

```
const std::vector<double>& getCamberLineX() const noexcept;
const std::vector<double>& getCamberLineY() const noexcept;
```

Description

Return the calculated camber-line coordinates.
The camber line runs from the leading edge towards the trailing edge on an ascending chordwise grid.

AirfoilData thickness-distribution getters

API level: Basic API

Signature

```
const std::vector<double>& getThicknessDistributionX() const noexcept;
const std::vector<double>& getThicknessDistributionY() const noexcept;
```

Description

Return the signed thickness distribution.
The distribution follows the complete contour order. Values on one side of the airfoil have a negative sign and values on the other side have a positive sign.

AirfoilData circulation and source getters

API level: Basic API

Signature

```
const std::vector<double>& getCirculationX() const noexcept;
const std::vector<double>& getCirculationY() const noexcept;
const std::vector<double>& getSourceX() const noexcept;
const std::vector<double>& getSourceY() const noexcept;
```

Description

Return the stored normalized circulation distribution and scaled source–sink distribution.

Method	Returned data
`getCirculationX`	Chordwise support coordinates of the circulation distribution.
`getCirculationY`	Normalized vortex-sheet strength $k^* = k/U_\infty$. In the standard property workflow, the distribution corresponds to $c_l = 0.5$.
`getSourceX`	Chordwise support coordinates of the scaled source–sink distribution.
`getSourceY`	Scaled source–sink distribution $q^* = q/(2U_\infty) = dz^{(t)}/dx$.

Notes

The circulation and source distributions generally contain different numbers of support

AirfoilData circulation and source getters *(continued)*

points.
Neither `getCirculationY` nor `getSourceY` returns a dimensional physical singularity strength.
For each stored element,

$$k_i = U_\infty \texttt{getCirculationY()[i]}, \qquad q_i = 2U_\infty \texttt{getSourceY()[i]}.$$

See Section 3 for the theoretical interpretation and normalization.

AirfoilData::getGradientData

API level: Basic API

Signature

```
GradientData getGradientData() const;
```

Description

Returns the upper- and lower-surface coordinate, first-derivative, and second-derivative vectors in one `GradientData` object.

Returns

A `GradientData` object containing copies of the stored vectors.

Notes

Check both `valid` and the required vector sizes before using the data.

Example

```
GradientData data = airfoil.getGradientData();

if (data.valid && !data.upperX.empty())
{
    std::cout << data.upperGradient[0] << "\n";
}
```

13.8 Saving Airfoil Coordinates

AirfoilData::saveAirfoilCoordinates

API level: Basic API

Signature

```
bool saveAirfoilCoordinates(
        const std::string& fileName,
        const std::string& source);
```

Description

Writes the current airfoil coordinates to a text file.
If the supplied file name does not end with `.dat`, the suffix is appended automatically.

AirfoilData::saveAirfoilCoordinates *(continued)*

The output file contains a comment header followed by either two or three numerical columns:

```
x y
```

or, if a thickness distribution of sufficient size is available:

```
x y yt
```

Parameters

`fileName`	Requested file name. The suffix `.dat` is added when necessary.
`source`	The `source` parameter is written unchanged into the file header after the text `Created by`. It may identify an application, calculation method, or processing step.

Returns

`true` if the file was opened and written successfully. Returns `false` if the object is not marked as created, the file cannot be opened, or a write error occurs.

Notes

The method requires:

```
airfoil.isCreated() == true
```

A successful call to `setAirfoilPoints` resets the created state to `false`. If the application regards the newly assigned contour as complete and ready for saving, it must explicitly mark the object:

```
if (!airfoil.setAirfoilPoints(x, y))
{
    return 1;
}

airfoil.setIsCreated(true);

if (!airfoil.saveAirfoilCoordinates("normalized_airfoil","normalization"))
{
    std::cerr << "Could not save airfoil.\n";
}
```

13.9 Version Information

AirfoilData version methods

API level: Basic API

Signature

```
static constexpr int versionMajor() noexcept;
static constexpr int versionMinor() noexcept;
static constexpr int versionPatch() noexcept;
static constexpr const char* versionString() noexcept;
```

AirfoilData version methods *(continued)*

Description

Return the compile-time version information of the `AirfoilData` library.

Method	Result for version 1.0.0
`versionMajor()`	`1`
`versionMinor()`	`0`
`versionPatch()`	`0`
`versionString()`	`"1.0.0"`

Example

```
std::cout << "AirfoilData "
<< AirfoilData::versionString()
<< "\n";
```

14 Advanced API Reference

The Advanced API exposes individual processing steps that are normally combined by the Basic API methods. It is intended for applications that require direct control over normalization, support-point distributions, geometric calculations, derivatives, normalized vortex-sheet distributions, or scaled source–sink distributions.

Advanced methods generally operate either on explicitly supplied coordinate vectors or on intermediate data already stored in an `AirfoilData` object.

Note

Most Advanced API methods return `bool`. Always check the returned value before using output parameters or modified object data.
Unless stated otherwise, input coordinate values must be finite and corresponding x and y vectors must have equal lengths.

14.1 Support-Point Management

AirfoilData::initAirfoil

API level: Advanced API

Signature

```
void initAirfoil(std::size_t size);
```

Description

Initializes internal airfoil, camber-line, and thickness-distribution vectors for a requested number of support points.
After validation and possible adjustment to an odd point count, the chordwise vectors are

AirfoilData::initAirfoil *(continued)*

initialized with the default half-cosine support-point distribution.

Parameters

`size`	Requested number of support points. The value must lie in the closed interval 99 to 511. An even value is reduced by one before the vectors are created.

Notes

If `size` lies outside the interval 99 to 511, the current airfoil and all derived data are cleared. The desired point-count setting remains unchanged.
If `size` is even, it is reduced to the next lower odd number. For example, `initAirfoil(100)` creates vectors containing 99 points and sets the desired point count to 99.
This method prepares storage but does not create a geometrically valid airfoil contour. The generated ordinate vectors contain zeros.

AirfoilData::createXDistribution

API level: Advanced API

Signature

```
static std::vector<double> createXDistribution(
        std::size_t pointCount);
static std::vector<double> createXDistribution(
        std::size_t pointCount,
        XDistributionMode mode);
```

Description

Generates a chordwise support-point distribution for a complete airfoil contour.
Both distributions run in contour order:

$$x = 1 \quad \longrightarrow \quad x = 0 \quad \longrightarrow \quad x = 1.$$

The overload without an explicit mode uses `XDistributionMode::HalfCosine`.

Parameters

`pointCount`	Number of support points. The value must lie in the closed interval 99 to 511.
`mode`	Selects the full-cosine or half-cosine distribution.

Returns

A vector containing the generated chordwise coordinates.
If `pointCount` is not within the valid range of 99 to 511, an empty vector is returned.

Notes

`XDistributionMode::Cosine` clusters support points near both the leading and trailing edges.
`XDistributionMode::HalfCosine` clusters points mainly near the leading edge and is used by the overload without an explicit mode.
Unlike `initAirfoil`, `setNumberOfDesiredAirfoilPoints`, and `redistributeAirfoil`,

AirfoilData::createXDistribution *(continued)*

this method does not modify an even point count. An even value inside the allowed range is accepted unchanged.
For operations requiring one distinct leading-edge support point, use an odd point count. An even complete-contour distribution can form a leading-edge plateau.

Example

```
std::vector<double> x =
AirfoilData::createXDistribution(
255,
XDistributionMode::Cosine);
```

AirfoilData desired point count

API level: Advanced API

Signature

```
std::size_t getNumberOfDesiredAirfoilPoints() const noexcept;
bool setNumberOfDesiredAirfoilPoints(std::size_t np);
```

Description

Get or set the desired target point count stored in the object.
The desired point count is an application setting. It is not necessarily equal to the number of points currently stored in the airfoil contour.

Parameters

`np`	Requested target point count. The value must lie between 99 and 511. An even value is reduced by one.

Returns

`setNumberOfDesiredAirfoilPoints` returns `true` if the supplied value lies within the supported range. It returns `false` for values below 99 or above 511, leaving the previous setting unchanged.

Notes

If an even value is accepted, the stored value is the next lower odd number. For example, a request for 100 points stores 99.
The default desired point count is 255.

AirfoilData support-point count getters

API level: Advanced API

Signature

AirfoilData support-point count getters *(continued)*

```
int getNumberOfCamberPoints() const noexcept;
int getNumberOfUpperGradientPoints() const noexcept;
int getNumberOfLowerGradientPoints() const noexcept;
int getNumberOfCirculationPoints() const noexcept;
int getNumberOfSourcePoints() const noexcept;
int getNumberOfThicknessDistributionPoints() const noexcept;
```

Description

Return the numbers of support points currently stored for selected calculated distributions.

Method	Counted data
`getNumberOfCamberPoints`	Camber-line support points.
`getNumberOfUpperGradientPoints`	Upper-surface derivative support points.
`getNumberOfLowerGradientPoints`	Lower-surface derivative support points.
`getNumberOfCirculationPoints`	Circulation support points.
`getNumberOfSourcePoints`	Support points of the scaled source–sink distribution.
`getNumberOfThicknessDistributionPoints`	Thickness-distribution support points.

Notes

These methods return the size of the corresponding x vector. They do not independently verify that the associated y vector has the same size.

14.2 Geometry Preparation and Transformation

AirfoilData::getNoseIndex

API level: Advanced API

Signature

```
bool getNoseIndex(
        const std::vector<double>& profileX,
        const std::vector<double>& profileY,
        NoseDetectionMode mode,
        std::size_t& noseIndex) const;
```

Description

Determines a leading-edge candidate in an airfoil contour.
With `NoseDetectionMode::LongestSecant`, the secant origin is the midpoint of the two contour endpoints:

$$x_{\mathrm{TE}} = x_0 + \frac{1}{2}(x_{n-1} - x_0), \qquad y_{\mathrm{TE}} = y_0 + \frac{1}{2}(y_{n-1} - y_0).$$

The method selects the interior contour point with the greatest Euclidean distance from this trailing-edge midpoint:

AirfoilData::getNoseIndex *(continued)*

$$i_{\mathrm{LE}} = \arg\max_{1 \le i \le n-2} \sqrt{(x_i - x_{\mathrm{TE}})^2 + (y_i - y_{\mathrm{TE}})^2}.$$

For a closed trailing edge, the two endpoint coordinates coincide and their midpoint is the trailing-edge point itself. For an open trailing edge, the midpoint lies between the lower and upper trailing-edge corners. This avoids a bias towards either corner.
With `NoseDetectionMode::LocalMinimum`, the method searches for a local minimum of the chordwise coordinate.

Parameters

`profileX`	Chordwise contour coordinates.
`profileY`	Corresponding vertical coordinates. Must have the same size as `profileX`.
`mode`	Leading-edge detection method.
`noseIndex`	Output parameter receiving the detected contour index.

Returns

`true` if the input vectors have equal lengths, contain at least two points, contain only finite values, and the selected search procedure completed without a non-finite distance calculation. Returns `false` for invalid input dimensions, non-finite coordinates, or a non-finite longest-secant calculation.

Notes

The method reports the result of the index-search procedure. It does not by itself guarantee that the returned index represents a geometrically valid interior leading edge.
In particular, an index value of zero may indicate that no strict local minimum or no positive-length interior secant candidate was selected. Geometry methods that call `getNoseIndex` normally apply an additional interior-leading-edge validation.
The output parameter should therefore be used together with the returned status and, where required, additional geometric checks.

AirfoilData::reorderData

API level: Advanced API

Signature

```
void reorderData(
        std::vector<double>& xCoordinates,
        std::vector<double>& yCoordinates);
```

Description

Reverses the order of the supplied coordinate vectors.
Each x/y coordinate pair remains associated because both vectors are reversed together.

Parameters

`xCoordinates`	Chordwise coordinates to reverse.
`yCoordinates`	Corresponding vertical coordinates to reverse.

Notes

AirfoilData::reorderData *(continued)*

If the vector lengths differ, the method performs no operation.
The method does not geometrically sort independent points. It only reverses the complete contour order.

AirfoilData::scale

API level: Advanced API

Signature

```
bool scale(
        const std::vector<double>& xIn,
        const std::vector<double>& yIn,
        std::vector<double>& xOut,
        std::vector<double>& yOut,
        double xMin,
        double xMax);
```

Description

Scales a coordinate set so that the minimum and maximum input chordwise coordinates are mapped onto the requested target interval.
If

$$L = \max(x_{\text{in}}) - \min(x_{\text{in}}), \qquad s = \frac{x_{\max} - x_{\min}}{L},$$

the transformed coordinates are:

$$x_{\text{out},i} = x_{\min} + (x_{\text{in},i} - \min(x_{\text{in}}))\, s,$$

$$y_{\text{out},i} = y_{\text{in},i} s.$$

Thus, the same scale factor is applied to both coordinate directions.

Parameters

`xIn`	Input chordwise coordinates.
`yIn`	Input vertical coordinates. Must have the same size as `xIn`.
`xOut`	Output chordwise coordinates. Must initially have the same size as the input vectors.
`yOut`	Output vertical coordinates. Must initially have the same size as the input vectors and must be a different vector from `xOut`.
`xMin`	Lower boundary of the target chordwise interval.
`xMax`	Upper boundary of the target chordwise interval.

Returns

`true` if the coordinate set was scaled successfully. Returns `false` for inconsistent dimensions, empty input, identical output vectors, non-finite input values, zero input extent, or non-finite results.

Notes

AirfoilData::scale *(continued)*

The operation is transactional with respect to the output vectors. They are replaced only after all scaled values have been calculated successfully.

AirfoilData::rotate

API level: Advanced API

Signature

```
bool rotate(
        const std::vector<double>& xIn,
        const std::vector<double>& yIn,
        std::vector<double>& xOut,
        std::vector<double>& yOut,
        double angleDeg);
```

Description

Rotates a coordinate set around the origin by an angle specified in degrees.
The public method converts the supplied angle to radians and then applies the transformation:

$$x' = x\cos(\alpha) + y\sin(\alpha),$$

$$y' = -x\sin(\alpha) + y\cos(\alpha),$$

where

$$\alpha = \texttt{angleDeg}\frac{\pi}{180}.$$

A positive angle corresponds to a clockwise rotation in the conventional Cartesian coordinate system.

Parameters

`xIn`	Input chordwise coordinates.
`yIn`	Input vertical coordinates. Must have the same size as `xIn`.
`xOut`	Output chordwise coordinates. Must initially have the same size as the input vectors.
`yOut`	Output vertical coordinates. Must initially have the same size as the input vectors and must be a different vector from `xOut`.
`angleDeg`	Rotation angle in degrees. Must be finite.

Returns

`true` if all coordinates were rotated successfully.
Returns `false` for inconsistent dimensions, aliased output vectors, non-finite input coordinates, a non-finite angle, or non-finite results.

Notes

The output vectors are replaced only after the complete transformation has succeeded.
The rotation is performed around the coordinate-system origin. If rotation around another point is required, translate the coordinates before the call and translate them back afterwards.

AirfoilData::rotate *(continued)*

Example

```
std::vector<double> rotatedX(x.size());
std::vector<double> rotatedY(y.size());

if (!airfoil.rotate(
x,
y,
rotatedX,
rotatedY,
30.0))
{
    std::cerr << "Rotation failed.\n";
}
```

AirfoilData::repairXHooks

API level: Advanced API

Signature

```
bool repairXHooks(
        std::vector<double>& xCoordinates,
        std::vector<double>& yCoordinates,
        std::size_t& removedCount);
```

Description

Removes small chordwise contour reversals, referred to as x-hooks, from an airfoil contour. The method determines the point with the global minimum x coordinate and treats it as the leading edge. It then requires:

- strictly decreasing x coordinates from the first point to the leading edge,
- strictly increasing x coordinates from the leading edge to the final point.

Points violating these conditions are removed together with their corresponding ordinate values.

Parameters

`xCoordinates`	Chordwise contour coordinates to inspect and possibly modify.
`yCoordinates`	Corresponding vertical coordinates.
`removedCount`	Output parameter receiving the number of removed points.

Returns

`true` if the contour is already monotone by branch or was repaired successfully.
Returns `false` if the input vectors are inconsistent, fewer than three points are supplied, or more than 20 violating points must be removed.

Notes

The method modifies the supplied vectors directly. It is intended for small local reversals produced from highly nonlinear airfoil generators (e.g. based upon conformal mapping at the trailing edge), not for general sorting or repair of self-intersecting contours.
If the repair limit is exceeded, the method returns `false`. Applications should not assume

AirfoilData::repairXHooks *(continued)*

that the vectors remain unchanged after such a failed repair attempt.

AirfoilData::mapAirfoilToSupportPoints

API level: Advanced API

Signature

```
bool mapAirfoilToSupportPoints(
        const std::vector<double>& inputX,
        const std::vector<double>& inputY,
        std::vector<double>& redistributedX,
        std::vector<double>& redistributedY,
        std::size_t newPointCount);
```

Description

Scales, orients, and maps an airfoil contour onto a newly generated support-point distribution using Akima interpolation.
The method:

1. scales the input coordinates relative to the chordwise range,
2. corrects the contour orientation where necessary,
3. removes consecutive duplicated chordwise values,
4. determines the leading edge,
5. transforms the lower-side chordwise coordinates to obtain one ascending interpolation grid,
6. generates new lower- and upper-side support points,
7. interpolates the ordinate values,
8. restores the ordinary positive chordwise coordinates.

Parameters

`inputX`	Input chordwise coordinates.
`inputY`	Corresponding vertical coordinates.
`redistributedX`	Output vector receiving the mapped chordwise coordinates.
`redistributedY`	Output vector receiving the mapped vertical coordinates. Must be a different vector from `redistributedX`.
`newPointCount`	Requested output point count. At least three points are required.

Returns

`true` if mapping succeeded. Returns `false` for inconsistent or non-finite input data, an invalid output-vector configuration, an invalid point count, an unusable leading edge, interpolation failure, or non-finite results.

Notes

The output vectors are replaced only after the complete operation has succeeded.

AirfoilData::normalizeAirfoil

API level: Advanced API

Signature

```
bool normalizeAirfoil(
        const std::vector<double>& xIn,
        const std::vector<double>& yIn,
        std::vector<double>& xOut,
        std::vector<double>& yOut,
        SortMode sort,
        InterpolationMode interpolate,
        double& rotatedAngle);
```

Description

Normalizes an airfoil contour to a chord length of one and aligns the geometric chord with the chordwise coordinate axis.
The principal processing steps are:

1. scale the supplied contour,
2. optionally correct its orientation,
3. determine the leading edge by the longest-secant method,
4. translate the leading-edge point to the origin,
5. determine the trailing-edge reference,
6. rotate the chord onto the horizontal axis,
7. scale the rotated contour again to $0 \leq x \leq 1$,
8. repair small chordwise contour reversals,
9. adjust the trailing-edge endpoints,
10. optionally map the contour onto supplied target support points.

The longest-secant calculation uses the midpoint of the first and last contour points as its origin. This supports both closed and open trailing edges without bias towards either trailing-edge corner.

Parameters

`xIn`	Input chordwise coordinates.
`yIn`	Input vertical coordinates. Must have the same size as `xIn`.
`xOut`	Output chordwise coordinates. In `InterpolationMode::Map`, the values initially stored in this vector define the target support-point distribution.
`yOut`	Output vertical coordinates. Must initially have the required output size.
`sort`	Selects whether the contour orientation may be corrected.
`interpolate`	Selects whether the contour is mapped onto the target support points supplied in `xOut`.
`rotatedAngle`	Output parameter receiving the performed rotation in degrees.

Returns

`true` if normalization succeeded. Returns `false` for inconsistent dimensions, too few points, non-finite input data, invalid output vectors, an unusable contour geometry, unrepairable chordwise reversals, interpolation failure, or non-finite results.

AirfoilData::normalizeAirfoil *(continued)*

Notes

At least 11 input points are required.
With `InterpolationMode::None`, the output vectors must initially have the same size as the input vectors.
With `InterpolationMode::Map`, at least `AirfoilDataConstants::kMinTargetSupportPoints` target points are required. In version 1.0.0, this minimum is three. The original values in `xOut` define the target support-point distribution.
The public 99–511 point-count restriction used by `createXDistribution`, `initAirfoil`, `setNumberOfDesiredAirfoilPoints`, and `redistributeAirfoil` is not applied by `normalizeAirfoil`.
The angle returned through `rotatedAngle` is expressed in degrees. This is consistent with the public `rotate` method, which also uses degrees.
On success, `xOut` and `yOut` are replaced together. The output vectors are not committed before all normalization steps succeed.

14.3 Camber, Thickness, and Area Calculations

AirfoilData::calculateAirfoilCamberAndThickness

API level: Advanced API

Signature

```
bool calculateAirfoilCamberAndThickness();
```

Description

Calculates the camber-line and signed thickness distributions from the airfoil contour stored in the object.
A default chordwise support-point distribution with the same number of points as the airfoil contour is generated. The lower and upper surface ordinates are obtained by Akima interpolation, and the camber and thickness values are calculated from their midpoint and difference.

Returns

`true` if both distributions were calculated successfully. Returns `false` if the contour dimensions are inconsistent, a leading edge cannot be found, the interpolation grid is invalid, or interpolation fails.

Notes

On success, the results are available through:

- `getCamberLineX`,
- `getCamberLineY`,
- `getThicknessDistributionX`,
- `getThicknessDistributionY`.

AirfoilData::extractCamberLine

API level: Advanced API

Signature

```
bool extractCamberLine(
        const std::vector<double>& xCoordinates,
        const std::vector<double>& yCoordinates,
        const std::vector<double>& xThickness,
        std::vector<double>& yThickness,
        std::vector<double>& yCamber,
        CamberDifferenceMode mode);
```

Description

Calculates signed thickness and camber ordinates at a supplied contour support-point distribution.
In `CamberDifferenceMode::Simple`, corresponding upper and lower ordinates are interpolated at equal chordwise positions. The camber is their midpoint and the signed thickness is half their difference.

Parameters

`xCoordinates`	Airfoil contour chordwise coordinates.
`yCoordinates`	Airfoil contour vertical coordinates.
`xThickness`	Chordwise support distribution for the output data. It must have the same size as the airfoil coordinate vectors.
`yThickness`	Output vector receiving the signed thickness ordinates.
`yCamber`	Output vector receiving the camber ordinates. Must be a different vector from `yThickness`.
`mode`	Camber and thickness definition.

Returns

`true` if the output distributions were calculated successfully. Returns `false` for inconsistent dimensions, aliased output vectors, an unusable leading edge, unsupported mode, interpolation failure, or non-finite results.

Notes

In version 1.0.0, only `CamberDifferenceMode::Simple` is implemented. `CamberDifferenceMode::Correct` returns `false`.
The output vectors are replaced only after complete success.

AirfoilData::calculateThickness

API level: Advanced API

Signature

AirfoilData::calculateThickness *(continued)*

```
bool calculateThickness(
        const std::vector<double>& xCoordinates,
        const std::vector<double>& yCoordinates,
        ThicknessMode thicknessMode,
        double givenLocation,
        double& thickness,
        double& thicknessAtLocation,
        double& thicknessLocation,
        SideMode sideMode);
```

Description

Calculates the maximum thickness of a complete airfoil contour or a one-sided coordinate set.
In `ThicknessMode::Absolute`, upper and lower ordinates are interpolated at equal chordwise positions. The maximum difference is reported as the airfoil thickness.
In `ThicknessMode::Projected`, the greatest absolute upper and lower ordinates are determined separately and added.

Parameters

`xCoordinates`	Chordwise coordinates.
`yCoordinates`	Corresponding vertical coordinates.
`thicknessMode`	Selects absolute or projected thickness.
`givenLocation`	Chordwise position at which the local thickness is requested. Must be finite and lie in $[0, 1]$.
`thickness`	Output parameter receiving the maximum thickness.
`thicknessAtLocation`	Output parameter receiving the thickness at `givenLocation` in two-sided mode.
`thicknessLocation`	Output parameter receiving the chordwise location of the maximum thickness.
`sideMode`	Selects complete two-sided or one-sided input.

Returns

`true` if the thickness calculation succeeded. Returns `false` for invalid dimensions, aliased output parameters, an invalid requested location, an unsupported mode combination, an unusable leading edge, interpolation failure, or non-finite results.

Notes

The three scalar output parameters must refer to different variables.
`ThicknessMode::Projected` requires `SideMode::TwoSided`. In projected mode, the returned thickness location is set to zero because a unique location is not defined by that calculation.
In one-sided absolute mode, the maximum absolute ordinate is returned.

AirfoilData::applyThickness

API level: Advanced API

Signature

AirfoilData::applyThickness *(continued)*

```
bool applyThickness(
        ThicknessJob job,
        std::vector<double>& xCoordinates,
        std::vector<double>& yCoordinates,
        double desiredThickness);
```

Description

Optionally scales all vertical coordinates so that the resulting airfoil has a requested maximum thickness.
With `ThicknessJob::FixedThickness`, the method first calculates the current thickness and then applies the factor:

$$s = \frac{t_{\text{desired}}}{t_{\text{current}}}.$$

The chordwise coordinates remain unchanged.

Parameters

`job`	Selects whether a fixed-thickness adjustment is performed.
`xCoordinates`	Chordwise contour coordinates. These are used for the thickness calculation but are not modified.
`yCoordinates`	Vertical contour coordinates to scale.
`desiredThickness`	Requested maximum thickness. Must be finite.

Returns

`true` if the operation succeeded. Returns `false` if the desired thickness is non-finite, the current thickness cannot be calculated, the current thickness is zero in fixed-thickness mode, or scaling produces non-finite results.

Notes

With `ThicknessJob::ScaleOnly`, the ordinate scaling factor remains one and `desiredThickness` does not change the coordinates.
The ordinate vector is replaced only after all scaled values have been calculated successfully.

AirfoilData::calculateArea

API level: Advanced API

Signature

```
bool calculateArea(
        const std::vector<double>& xCoordinates,
        const std::vector<double>& yCoordinates,
        double& calculatedArea);
```

Description

Numerically calculates the cross-sectional area enclosed by an airfoil contour.
The method maps the contour onto a dense cosine-distributed support grid using Akima interpolation and sums rectangular and triangular area contributions between corresponding upper and lower points.

AirfoilData::calculateArea *(continued)*

Parameters

`xCoordinates`	Airfoil contour chordwise coordinates.
`yCoordinates`	Corresponding vertical coordinates.
`calculatedArea`	Output parameter receiving the calculated area.

Returns

`true` if the area was calculated successfully. Returns `false` for inconsistent coordinate vectors, an unusable leading edge, interpolation failure, invalid support-point data, or a non-finite result.

Notes

The internal integration grid contains `AirfoilDataConstants::kAreaSupportPoints` points. In version 1.0.0, this value is 32768.
For coordinates normalized to a chord length of one, the returned value is a dimensionless area relative to the square of the chord length.

AirfoilData::calculateDyNose

API level: Advanced API

Signature

```
bool calculateDyNose(
        const std::vector<double>& xCoordinates,
        const std::vector<double>& yCoordinates,
        double& calculatedNoseHeight);
```

Description

Calculates the upper-surface ordinate at the standard chordwise nose location:

$$x/c = \texttt{AirfoilDataConstants::kNoseLocation} = 0.0125.$$

Parameters

`xCoordinates`	Airfoil contour chordwise coordinates.
`yCoordinates`	Corresponding vertical coordinates.
`calculatedNoseHeight`	Output parameter receiving the interpolated upper-surface ordinate.

Returns

`true` if the upper surface was identified and interpolation succeeded. Returns `false` for inconsistent coordinate vectors, invalid leading-edge detection, interpolation failure, or a non-finite result.

AirfoilData::calculateTeThickness

API level: Advanced API

Signature

AirfoilData::calculateTeThickness *(continued)*

```
void calculateTeThickness();
```

Description

Calculates the vertical trailing-edge thickness from the first and last contour ordinates:

$$t_{\mathrm{TE}} = y_{n-1} - y_0.$$

The result is stored in the object and can be retrieved with `getTrailingEdgeThickness`.

Notes

If no consistent airfoil coordinate data are available, the method emits an error message and leaves the existing stored value unchanged.

AirfoilData::calculateAirfoilGeometryData

API level: Advanced API

Signature

```
bool calculateAirfoilGeometryData();
```

Description

Calculates the principal scalar geometric properties from the current airfoil contour and an already available camber line.
The method calculates:

- maximum thickness and its location,
- characteristic nose height,
- cross-sectional area,
- signed maximum camber and its location,
- trailing-edge thickness.

Returns

`true` if all geometry calculations succeeded. Returns `false` if a required distribution is missing or inconsistent, or if any individual calculation fails.

Notes

The camber and thickness distributions must already have been calculated, for example with:

```
airfoil.calculateAirfoilCamberAndThickness();
```

Scalar geometry members are committed only after all principal calculations have succeeded.

14.4 Surface Separation and Derivatives

AirfoilData::calculateUpperLowerCoordinates

API level: Advanced API

Signature

```
bool calculateUpperLowerCoordinates();
```

Description

Separates the current complete airfoil contour into lower- and upper-surface coordinate vectors.
The leading-edge point is included in both surface data sets. The upper surface is stored in reversed contour order so that its chordwise coordinates run from the leading edge towards the trailing edge.

Returns

`true` if a valid interior leading edge was found and the surfaces were separated successfully. Returns `false` otherwise.

Notes

The results are available through:

- `getLowerSurfaceX`,
- `getLowerSurfaceY`,
- `getUpperSurfaceX`,
- `getUpperSurfaceY`.

AirfoilData::calculateDerivatives

API level: Advanced API

Signature

```
bool calculateDerivatives(
      const std::vector<double>& xCoordinates,
      const std::vector<double>& yCoordinates,
      std::vector<double>& firstLower,
      std::vector<double>& secondLower,
      std::vector<double>& thirdLower,
      std::vector<double>& firstUpper,
      std::vector<double>& secondUpper,
      std::vector<double>& thirdUpper,
      SideMode sideMode);
```

Description

Calculates first and second chordwise derivatives on one or both sides of a coordinate set. Akima interpolation is used to obtain values at locally symmetric chordwise distances. The derivatives are then approximated by centered finite-difference expressions. Endpoint values are extrapolated from neighboring derivative values.

Parameters

AirfoilData::calculateDerivatives *(continued)*

`xCoordinates`	Input chordwise coordinates.
`yCoordinates`	Corresponding vertical coordinates.
`firstLower`	Output vector receiving lower-side first derivatives.
`secondLower`	Output vector receiving lower-side second derivatives.
`thirdLower`	Reserved output vector for lower-side third derivatives. It must be empty.
`firstUpper`	Output vector receiving upper-side first derivatives.
`secondUpper`	Output vector receiving upper-side second derivatives.
`thirdUpper`	Reserved output vector for upper-side third derivatives. It must be empty.
`sideMode`	Selects one-sided or two-sided processing.

Returns

`true` if the derivatives were calculated successfully. Returns `false` for inconsistent dimensions, aliased output vectors, insufficient support points, an invalid leading-edge split, interpolation failure, non-finite results, or a request for third derivatives.

Notes

All six output vectors must be distinct objects.
Third derivatives are not implemented in version 1.0.0. To request the available first and second derivatives, pass empty vectors for `thirdLower` and `thirdUpper`.
In `SideMode::TwoSided`, at least four support points are required on each side of the leading edge.
In `SideMode::OneSided`, at least four points are required and the principal derivative results are returned in the upper-side output vectors.
The quantities named *curvature* elsewhere in the API correspond to the second derivatives returned here.

AirfoilData::calculateGradientCurvature

API level: Advanced API

Signature

```
bool calculateGradientCurvature();
```

Description

Calculates and stores first and second chordwise derivatives for the upper and lower surfaces of the current airfoil contour.
Internally, the method calls `calculateDerivatives` in two-sided mode and associates the resulting derivative vectors with their surface chordwise coordinates.

Returns

`true` if the leading edge was identified and all derivative data were calculated successfully. Returns `false` otherwise.

Notes

The results are available through:

AirfoilData::calculateGradientCurvature *(continued)*

- `getGradientDataUpperX`,
- `getGradientDataLowerX`,
- `getGradientDataUpper`,
- `getGradientDataLower`,
- `getCurvatureDataUpper`,
- `getCurvatureDataLower`,
- `getGradientData`.

The values returned as curvature data are second chordwise derivatives d^2y/dx^2, not differential-geometric curvature values.

AirfoilData derivative-data getters

API level: Advanced API

Signature

```
const std::vector<double>& getGradientDataUpperX() const noexcept;
const std::vector<double>& getGradientDataLowerX() const noexcept;
const std::vector<double>& getGradientDataUpper() const noexcept;
const std::vector<double>& getGradientDataLower() const noexcept;
const std::vector<double>& getCurvatureDataUpper() const noexcept;
const std::vector<double>& getCurvatureDataLower() const noexcept;
```

Description

Return constant references to the separately stored upper- and lower-surface derivative vectors.

Method group	Contents
`getGradientDataUpperX`, `getGradientDataLowerX`	Chordwise support coordinates.
`getGradientDataUpper`, `getGradientDataLower`	First derivatives dy/dx.
`getCurvatureDataUpper`, `getCurvatureDataLower`	Second derivatives d^2y/dx^2.

14.5 Normalized Vortex-Sheet and Scaled Source–Sink Calculations

AirfoilData::calculateCirculation

API level: Advanced API

Signature

```
bool calculateCirculation(
        const std::vector<double>& camberLineX,
        const std::vector<double>& camberLineY,
        std::vector<double>& xCirculation,
        std::vector<double>& circulationStrength,
        double angleOfAttack);
```

Description

AirfoilData::calculateCirculation *(continued)*

Calculates the normalized inviscid vortex-sheet distribution from a camber line using thin-airfoil theory.
The values written to `circulationStrength` represent:

$$k^*(x) = \frac{k(x)}{U_\infty}.$$

Using the Fourier coefficients of the camber-line slope, the normalized distribution is evaluated as:

$$\frac{k(\varphi)}{U_\infty} = 2\left[A_0 \tan\left(\frac{\varphi}{2}\right) + \sum_{n=1}^{N} A_n \sin(n\varphi)\right].$$

The method also calculates the inviscid geometric zero-lift angle and pitching-moment coefficient stored in the object.

Parameters

`camberLineX`	Ascending camber-line chordwise coordinates. At least four points are required, and all values must lie in $[0, 1]$.
`camberLineY`	Corresponding camber-line ordinates.
`xCirculation`	Output vector receiving the circulation support coordinates.
`circulationStrength`	Output vector receiving the normalized vortex-sheet strength k/U_∞. Must be a different vector from `xCirculation`.
`angleOfAttack`	Angle of attack in degrees.

Returns

`true` if the Fourier coefficients and circulation distribution were calculated successfully. Returns `false` for invalid dimensions, insufficient points, aliased output vectors, an invalid angle, invalid chordwise coordinates, derivative failure, or non-finite results.

Notes

On success, the method also updates:

- `getAlpha0Geo()`,
- `getCm0Geo()`.

The angle passed to this method is expressed in degrees, consistent with the public `rotate` method and the angle returned by `normalizeAirfoil`.

The returned circulation distribution is a normalized circulation density, not a dimensional circulation. If the free-stream velocity is known, the dimensional vortex-sheet strength can be reconstructed as:

$$k = U_\infty \, \texttt{circulationStrength}.$$

See Section 3.2 and Ref. [1].

In version 1.0.0, `AirfoilDataConstants::kCirculationFourierCount` is 101, and `AirfoilDataConstants::kCirculationPhaseGrid` is 2000.

AirfoilData::calculateCirculationCl05

API level: Advanced API

Signature

```
bool calculateCirculationCl05(
        const std::vector<double>& xCoordinates,
        const std::vector<double>& yCoordinates,
        std::vector<double>& xCirculation,
        std::vector<double>& circulationStrength);
```

Description

Calculates the normalized inviscid vortex-sheet distribution at the fixed reference lift coefficient

$$c_l = 0.5.$$

The method first determines the geometric zero-lift angle from the camber line. It then calculates the angle of attack corresponding to $c_l = 0.5$ from:

$$c_l = 2\pi(\alpha - \alpha_0),$$

and therefore:

$$\alpha = \alpha_0 + \frac{0.5}{2\pi}.$$

The angles in these equations are expressed in radians. The calculated angle is converted to degrees before being passed to `calculateCirculation`.
A second call then calculates the normalized distribution

$$k^* = \frac{k}{U_\infty}$$

at this reference condition.

Parameters

`xCoordinates`	Ascending camber-line chordwise coordinates.
`yCoordinates`	Corresponding camber-line ordinates.
`xCirculation`	Output vector receiving the circulation support coordinates.
`circulationStrength`	Output vector receiving the normalized vortex-sheet strength k/U_∞ for $c_l = 0.5$. It must be a different vector from `xCirculation`.

Returns

`true` if both circulation calculations succeeded. Returns `false` for invalid input, aliased output vectors, numerical failure, or a non-finite calculated angle of attack.

Notes

This is the circulation distribution used by `calculateAirfoilProperties` and stored by `updateCirculation`.

The fixed reference quantity is the lift coefficient $c_l = 0.5$, not the angle of attack.

AirfoilData::calculateCirculationCl05 *(continued)*

The standardized lift condition makes the distribution useful as a geometry descriptor and as input for circulation-based airfoil generation. The corresponding generator is not implemented in this library.

AirfoilData::updateCirculation

API level: Advanced API

Signature

```
bool updateCirculation(RecalcMode reCalc);
```

Description

Updates the circulation distribution stored in the object.

Parameters

`reCalc`	Selects whether the camber and thickness distributions are recalculated before the circulation update.

Returns

`true` if the required camber data were available and the circulation distribution was calculated successfully. Returns `false` otherwise.

Notes

With `RecalcMode::Recompute`, the previous object state is restored if the update fails. With `RecalcMode::Keep`, the method expects a valid camber line to be already stored. The result is available through `getCirculationX` and `getCirculationY`. It represents the normalized vortex-sheet strength k/U_∞ at the reference lift coefficient $c_l = 0.5$.

AirfoilData::calculateSourceStrength

API level: Advanced API

Signature

```
bool calculateSourceStrength(
        const std::vector<double>& xThickness,
        const std::vector<double>& yThickness,
        std::vector<double>& xq,
        std::vector<double>& q);
```

Description

Calculates the scaled source–sink distribution associated with a one-sided half-thickness distribution.
The method stores the first chordwise derivative:

$$q^*(x) = \frac{dz^{(t)}}{dx}.$$

In the classical thickness formulation, the physical source strength is

AirfoilData::calculateSourceStrength *(continued)*

$$q_{\text{physical}}(x) = 2U_\infty \frac{dz^{(t)}}{dx}.$$

Therefore, the output of this method satisfies:

$$q^* = \frac{q_{\text{physical}}}{2U_\infty}.$$

The scaled quantity is used because the distribution serves primarily as a geometric basis for source-based airfoil generation rather than as a dimensional aerodynamic result.

Parameters

`xThickness`	Ascending chordwise coordinates of the one-sided half-thickness distribution.
`yThickness`	Corresponding half-thickness ordinates $z^{(t)}$.
`xq`	Output vector receiving the chordwise support coordinates of the scaled distribution.
`q`	Output vector receiving $q^* = dz^{(t)}/dx$. Must be a different vector from `xq`.

Returns

`true` if all derivatives and scaled source values were calculated successfully. Returns `false` for aliased output vectors, invalid support data, derivative failure, insufficient derivative output, or non-finite results.

Notes

This method expects a one-sided half-thickness distribution, normally the ascending section from the leading edge to the trailing edge.
The output `q` is not the dimensional physical source strength. To reconstruct the latter, use:

$$q_{\text{physical}} = 2U_\infty q.$$

The normalized physical source strength is:

$$\frac{q_{\text{physical}}}{U_\infty} = 2q.$$

The output vectors are replaced only after successful completion.
See Section 3.1 and Ref. [2] for the theoretical background.

AirfoilData::updateSourceStrength

API level: Advanced API

Signature

```
bool updateSourceStrength(RecalcMode reCalc);
```

Description

Updates the scaled source–sink distribution stored in the object.
The method identifies the leading edge in the signed thickness distribution, extracts the ascending one-sided half-thickness section, and calls `calculateSourceStrength`.

AirfoilData::updateSourceStrength *(continued)*

The resulting values satisfy:

$$\texttt{sourceY} = \frac{dz^{(t)}}{dx} = \frac{q_{\text{physical}}}{2U_\infty}.$$

Parameters

`reCalc`	Selects whether the camber and thickness distributions are recalculated before the source-strength update.

Returns

`true` if a valid thickness distribution was available and the source-strength calculation succeeded. Returns `false` otherwise.

Notes

With `RecalcMode::Recompute`, the previous object state is restored if the update fails.
With `RecalcMode::Keep`, a valid thickness distribution must already be stored.

The result is available through `getSourceX` and `getSourceY`. It represents the geometry-scaled distribution

$$q^* = \frac{q_{\text{physical}}}{2U_\infty} = \frac{dz^{(t)}}{dx},$$

not a dimensional physical source strength.

AirfoilData::updateCirculationAndSourceStrength

API level: Advanced API

Signature

```
bool updateCirculationAndSourceStrength();
```

Description

Recalculates the camber and thickness distributions and then updates both the normalized vortex-sheet and scaled source–sink distributions.

Returns

`true` if all three processing stages succeeded. Returns `false` if camber/thickness extraction, circulation calculation, or source-strength calculation fails.

Notes

The operation is transactional. If any stage fails, the complete previous object state is restored.
This method is useful when the contour has changed but a complete recalculation of all remaining geometry data is not required.

14.6 Advanced Workflow Examples

The following examples summarize typical combinations of Advanced API methods.

Normalize a Contour

```
std::vector<double> normalizedX(x.size());
std::vector<double> normalizedY(y.size());
double rotationDegrees = 0.0;

if (!airfoil.normalizeAirfoil(
    x,
    y,
    normalizedX,
    normalizedY,
    SortMode::Sort,
    InterpolationMode::None,
    rotationDegrees))
{
    std::cerr << "Normalization failed.\n";
}
```

Rotate a Coordinate Set

```
std::vector<double> rotatedX(x.size());
std::vector<double> rotatedY(y.size());

if (!airfoil.rotate(
    x,
    y,
    rotatedX,
    rotatedY,
    5.0))
{
    std::cerr << "Rotation failed.\n";
}
```

The angle `5.0` is interpreted as five degrees. Positive angles rotate the coordinates clockwise.

Calculate Selected Geometric Values

```
double thickness = 0.0;
double thicknessAtHalfChord = 0.0;
double thicknessLocation = 0.0;

if (!airfoil.calculateThickness(
    x,
    y,
    ThicknessMode::Absolute,
    0.5,
    thickness,
    thicknessAtHalfChord,
    thicknessLocation,
    SideMode::TwoSided))
{
    std::cerr << "Thickness calculation failed.\n";
}
```

Update Potential-Flow Distributions

```
if (!airfoil.updateCirculationAndSourceStrength())
{
    std::cerr
    << "Circulation/source update failed.\n";
}
```

Note

For ordinary use, prefer the Basic API method `calculateAirfoilProperties`. Use the individual Advanced API methods when an application requires control over a specific processing stage.

15 Integration API Reference

The Integration API supports the use of `AirfoilData` inside larger applications. It provides facilities for:

- redirecting library messages to an application callback,
- suppressing or increasing diagnostic output,
- managing application-level object state,
- importing externally calculated aerodynamic data,
- storing pressure-coefficient distributions,
- storing derivative and conformal-mapping data,
- directly replacing individual internal data fields.

These methods are useful when `AirfoilData` is used as the central data container of a graphical or numerical application.

Note

Several Integration API setters replace individual data fields without checking the consistency of related fields. For example, `setPointsX` and `setPointsY` operate independently. For ordinary coordinate input, use the Basic API method `setAirfoilPoints`, which assigns both airfoil coordinate vectors together and verifies that their dimensions agree.

15.1 Diagnostic Output and Callbacks

By default, the library writes informational and normal messages to `std::cout`, while warnings and errors are written to `std::cerr`.

Applications can replace this console fallback with a callback function. This is useful for redirecting messages to a graphical user interface, log file, message collector, or application-specific reporting system.

AirfoilData::PrintMethod

Description

Defines the function-pointer type used for application-specific output handling.

Signature

```
using PrintMethod = void (*)(
        void* context,
        const std::string& text,
        AirfoilDataMessageType type);
```

The callback receives:

- an application-defined context pointer,
- the message text,
- the message severity or category.

Notes

The callback is a plain C++ function pointer. It may refer to a free function or a static member function. Capturing lambdas and non-static member functions cannot be assigned directly.
The library does not own the object referenced by `context`. The calling application must ensure that the context remains valid for as long as the callback can be invoked.

AirfoilData::setOutputMethod

API level: Integration API

Signature

```
void setOutputMethod(
        PrintMethod method,
        void* context);
```

Description

Installs an application-defined output callback and its associated context pointer.
When a callback is installed, messages emitted by the object are passed to the callback instead of being written to the standard output streams.

Parameters

`method`	Function pointer to the application-specific output handler. Passing `nullptr` restores the console fallback.
`context`	Application-defined pointer passed unchanged to every callback invocation. The pointer may be `nullptr` if the callback does not require context data.

Notes

The callback and context are stored as non-owning pointers. The application remains responsible for their lifetime.
Installing an output callback affects only the particular `AirfoilData` object on which the method is called.

Example

AirfoilData::setOutputMethod *(continued)*

```
#include <iostream>
#include <string>
#include <vector>

static void collectMessage(
void* context,
const std::string& text,
AirfoilDataMessageType type)
{
    std::vector<std::string>* messages = static_cast<std::vector<std::string>*>(context);


    if (messages != nullptr)
    {
        messages->push_back(text);
    }

    if (type == AirfoilDataMessageType::Error ||
        type == AirfoilDataMessageType::ErrorSevere)
    {
        std::cerr << text << "\n";
    }
}

// ...

std::vector<std::string> messages;

airfoil.setOutputMethod(
&collectMessage,
static_cast<void*>(&messages));
```

AirfoilData output callback getters

API level: Integration API

Signature

```
PrintMethod outputMethod() const;
void* outputContext() const;
```

Description

Return the currently installed output callback and context pointer.

Returns

`outputMethod` returns the stored function pointer. It returns `nullptr` if no callback is installed.
`outputContext` returns the stored application context pointer.

Notes

The returned context pointer remains owned by the calling application.

AirfoilData::emitOutput

API level: Integration API

Signature

```
void emitOutput(
        const std::string& text,
        AirfoilDataMessageType type);
```

Description

Emits a message through the central output mechanism of the object.
If output is suppressed, the method performs no action.
If a callback is installed, the callback receives the message. Otherwise, the method writes the message to a standard stream with a textual prefix.

Parameters

`text`	Message text.
`type`	Message category or severity.

Notes

The console fallback writes the following message types to `std::cerr`:

- `Warning`,
- `WarningSevere`,
- `Error`,
- `ErrorSevere`.

`Normal` and `Info` messages are written to `std::cout`.
This public method allows other library components or application adapters to use the same output path as `AirfoilData`.

AirfoilData output suppression

API level: Integration API

Signature

```
void setOutputSuppressed(bool suppress);
bool outputSuppressed() const;
```

Description

Enable or disable all messages emitted through `emitOutput`.

Parameters

`suppress`	Pass `true` to suppress output or `false` to enable it.

Returns

`outputSuppressed` returns the current suppression state.

Notes

Suppression affects both the application callback and the console fallback. Calculation

AirfoilData output suppression *(continued)*

results and return values are not affected.
Output suppression is useful when expected error paths are evaluated programmatically or when a calling application provides its own higher-level reporting.

AirfoilData::setVerbosityLevel

API level: Integration API

Signature

```
void setVerbosityLevel(std::size_t level);
```

Description

Sets the amount of informational diagnostic output generated by selected geometry methods.

Parameters

`level`	Requested diagnostic level. Supported levels are zero to three. Values greater than three are clamped to three.

Level	Meaning
0	No optional diagnostic output.
1	General processing information.
2	More detailed processing and interpolation information.
3	Highest supported diagnostic level.

Notes

The precise number of messages depends on the method being executed. Error and warning messages are not controlled solely by the verbosity level; use `setOutputSuppressed` to suppress all output.
There is no public getter for the current verbosity level in version 1.0.0.

15.2 Application-Level Object State

AirfoilData created state

API level: Integration API

Signature

```
void setIsCreated(bool value);
bool isCreated() const noexcept;
```

Description

Set or query the application-level created state of the airfoil object.

Parameters

`value`	New created state.

AirfoilData created state *(continued)*

Returns

`isCreated` returns the currently stored state.

Notes

The created state is separate from the existence of coordinate vectors and from the success of geometry calculations. It indicates that the application regards the object as a created and saveable airfoil.
In particular, `saveAirfoilCoordinates` returns `false` unless `isCreated()` is `true`.
A successful call to `setAirfoilPoints` resets the created state to `false`, because all data derived from the previous contour are cleared. An application that has loaded or constructed a complete airfoil may explicitly set the state afterwards:

```
if (airfoil.setAirfoilPoints(x, y))
{
    airfoil.setIsCreated(true);
}
```

Clearing calculated geometry data resets the created state to `false`.

15.3 External Aerodynamic Data

The following methods store aerodynamic results calculated by another solver or application component. They do not calculate viscous aerodynamic coefficients themselves.

AirfoilData::setAerodynamicData

API level: Integration API

Signature

```
bool setAerodynamicData(
     const std::vector<double>& alpha,
     const std::vector<double>& cD,
     const std::vector<double>& cDf,
     const std::vector<double>& cDp,
     const std::vector<double>& cL,
     const std::vector<double>& cM,
     const AeroScalars& scalars,
     const std::vector<double>& eCurrent,
     const std::vector<bool>& converged);
```

Description

Stores a complete set of externally calculated aerodynamic data.
All coefficient, efficiency, and convergence vectors must contain one element for every angle of attack. The scalar values are supplied together in an `AeroScalars` structure.

Parameters

AirfoilData::setAerodynamicData *(continued)*

`alpha`	Angles of attack. The unit is determined by the supplying application; degrees are recommended.
`cD`	Total drag coefficients.
`cDf`	Friction-drag coefficients.
`cDp`	Pressure-drag coefficients.
`cL`	Lift coefficients.
`cM`	Pitching-moment coefficients.
`scalars`	Scalar aerodynamic values: `alpha0`, `cl0`, `cd0`, `cm0`, `clMax`, and `eMax`.
`eCurrent`	Efficiency or glide-ratio values corresponding to the operating points.
`converged`	Solver convergence state for every operating point.

Returns

`true` if all vector dimensions and required numerical values are valid and the complete data set was stored.
Returns `false` if:

- a coefficient, efficiency, or convergence vector has a different length from `alpha`,
- one of the members of `scalars` is non-finite,
- a converged operating point contains a non-finite vector value.

Notes

For entries whose corresponding `converged` value is `true`, all values in `alpha`, `cD`, `cDf`, `cDp`, `cL`, `cM`, and `eCurrent` must be finite.
Entries marked as not converged may contain non-finite placeholder values. Applications must inspect `getConverged` before using an operating point.
All six scalar members of `AeroScalars` must be finite regardless of the convergence-vector contents.
The vector data are copied into temporary storage and committed together after validation. If validation fails, the previously stored aerodynamic data remain unchanged.

Example

```
AeroScalars scalars;
scalars.alpha0 = -1.5;
scalars.cl0 = 0.11;
scalars.cd0 = 0.009;
scalars.cm0 = -0.04;
scalars.clMax = 1.2;
scalars.eMax = 35.0;

if (!airfoil.setAerodynamicData(
alpha,
cd,
cdf,
cdp,
cl,
cm,
scalars,
efficiency,
```

AirfoilData::setAerodynamicData *(continued)*

```
converged))
{
    std::cerr
    << "Aerodynamic data are inconsistent.\n";
}
```

AirfoilData aerodynamic data getters

API level: Integration API

Signature

```
const std::vector<double>& getAlphas() const noexcept;
const std::vector<double>& getCl() const noexcept;
const std::vector<double>& getCd() const noexcept;
const std::vector<double>& getCdf() const noexcept;
const std::vector<double>& getCdp() const noexcept;
const std::vector<double>& getCm() const noexcept;
const std::vector<double>& getE() const noexcept;
const std::vector<bool>& getConverged() const noexcept;
double getAlpha0() const noexcept;
double getCl0() const noexcept;
double getCd0() const noexcept;
double getCm0() const noexcept;
double getClMax() const noexcept;
double getEMax() const noexcept;
```

Description

Return the externally stored aerodynamic coefficient, efficiency, convergence, and scalar result data.

Method	Returned data
`getAlphas`	Angles of attack.
`getCl`	Lift coefficients.
`getCd`	Total drag coefficients.
`getCdf`	Friction-drag coefficients.
`getCdp`	Pressure-drag coefficients.
`getCm`	Pitching-moment coefficients.
`getE`	Efficiency or glide-ratio values for the individual operating points.
`getConverged`	Convergence state for each operating point.
`getAlpha0`	Externally stored zero-lift angle.
`getCl0`	Externally stored lift coefficient at zero angle of attack.
`getCd0`	Externally stored drag coefficient at the reference condition.
`getCm0`	Externally stored pitching-moment coefficient at the reference condition.
`getClMax`	Externally stored maximum lift coefficient.
`getEMax`	Externally stored maximum efficiency or glide ratio.

Notes

The externally supplied values returned by `getAlpha0` and `getCm0` are distinct from the inviscid geometric values returned by:

AirfoilData aerodynamic data getters *(continued)*

- `getAlpha0Geo`,
- `getCm0Geo`.

The `Geo` values are calculated internally from the camber line using thin-airfoil theory. The remaining scalar values in this box are imported through `setAerodynamicData`.
Always inspect `getConverged()` before using the vector values of an operating point. Entries marked as not converged may contain non-finite placeholders.

15.4 Pressure-Coefficient Data

Pressure-coefficient distributions can be stored for several operating points. The outer vector normally represents the angle-of-attack or operating-point index; each inner vector contains one surface distribution.

AirfoilData pressure-data setters

API level: Integration API

Signature

```
void setCpUpper(
        const std::vector<std::vector<double>>& values);
void setCpLower(
        const std::vector<std::vector<double>>& values);
void setCpUpperX(const std::vector<double>& values);
void setCpLowerX(const std::vector<double>& values);
```

Description

Store upper- and lower-surface pressure-coefficient distributions and their corresponding chordwise coordinates.

Parameters

`values`	Pressure-coefficient matrix or chordwise support-point vector to store.

Notes

These methods copy the supplied data and do not validate the correspondence between:

- the number of operating points,
- the number of angles of attack,
- the lengths of individual pressure distributions,
- the associated upper or lower x coordinate vectors.

The integrating application is responsible for maintaining these relationships.

AirfoilData pressure-data getters

API level: Integration API

Signature

AirfoilData pressure-data getters *(continued)*

```
const std::vector<std::vector<double»& getUpperCp() const noexcept;
const std::vector<std::vector<double»& getLowerCp() const noexcept;
const std::vector<double>& getCpUpperX() const noexcept;
const std::vector<double>& getCpLowerX() const noexcept;
```

Description

Return the stored pressure-coefficient matrices and their surface support coordinates.

Returns

Constant references to the internally stored data.

Notes

No validity flag is associated with these data. The calling application must know whether the distributions have been populated and are dimensionally consistent.

15.5 Low-Level Geometry Setters

The following methods independently replace internal geometry fields. They are intended primarily for application adapters, data import, compatibility, and specialized processing pipelines.

AirfoilData::setPointsX / setPointsY

API level: Integration API

Signature

```
void setPointsX(const std::vector<double>& values);
void setPointsY(const std::vector<double>& values);
```

Description

Independently replace one coordinate vector of the stored airfoil contour.

Parameters

`values`	Coordinate vector to copy into the object.

Notes

For ordinary airfoil input, prefer:

```
airfoil.setAirfoilPoints(x, y);
```

The individual setters cannot verify that the resulting x/y pair has equal dimensions. They also do not validate coordinate finiteness, contour order, or airfoil geometry.
Replacing only one vector can temporarily or permanently make the object inconsistent. Previously calculated data may no longer correspond to the stored contour after either setter is called.

AirfoilData camber and thickness setters

API level: Integration API

Signature

```
void setCamberLineX(const std::vector<double>& values);
void setCamberLineY(const std::vector<double>& values);
void setThicknessX(const std::vector<double>& values);
void setThicknessY(const std::vector<double>& values);
```

Description

Independently replace the stored camber-line or thickness-distribution coordinate vectors.

Parameters

`values`	Vector to copy into the selected data field.

Notes

The methods do not check that the corresponding x and y vectors have equal dimensions or that the data are numerically valid.
Data inserted through these setters can be used by advanced update methods such as `updateCirculation` or `updateSourceStrength`, provided all required invariants are satisfied.

AirfoilData surface-coordinate setters

API level: Integration API

Signature

```
void setPointsUpperX(const std::vector<double>& values);
void setPointsUpperY(const std::vector<double>& values);
void setPointsLowerX(const std::vector<double>& values);
void setPointsLowerY(const std::vector<double>& values);
```

Description

Independently replace the stored upper- or lower-surface coordinate vectors.

Parameters

`values`	Surface coordinate vector to copy into the object.

Notes

The methods do not validate dimensional consistency between the corresponding x and y vectors.
Ordinarily, these vectors are generated by `calculateUpperLowerCoordinates` as part of the standard property calculation.

AirfoilData scalar geometry setters

API level: Integration API

Signature

```
void setThickness(double value);
void setTeThickness(double value);
void setThickLocation(double value);
void setCamber(double value);
void setCamberLocation(double value);
```

Description

Directly replace selected scalar geometry results stored in the object.

Method	Modified value
`setThickness`	Maximum airfoil thickness.
`setTeThickness`	Trailing-edge thickness.
`setThickLocation`	Chordwise location of maximum thickness.
`setCamber`	Signed maximum camber.
`setCamberLocation`	Chordwise location of maximum camber.

Notes

These methods do not validate finiteness, physical ranges, or consistency with the stored contour and distributions.
They are intended for importing externally calculated values or adapting existing application data. Ordinary users should obtain these values through the calculation methods.

15.6 Imported Derivative Data

AirfoilData derivative-data setters

API level: Integration API

Signature

```
void setPointsCamberDerivativeX(
        const std::vector<double>& values);
void setPointsCamberDerivativeY(
        const std::vector<double>& values);
void setPointsThicknessDerivativeX(
        const std::vector<double>& values);
void setPointsThicknessDerivativeY(
        const std::vector<double>& values);
```

Description

Store externally calculated camber-line or thickness-distribution derivative data.
The `X` vectors contain support coordinates. The corresponding `Y` vectors contain derivative values.

Parameters

AirfoilData derivative-data setters *(continued)*

`values`	Support-point or derivative vector to copy into the selected field.

Notes

These methods do not calculate derivatives and do not validate matching dimensions.

AirfoilData imported derivative-data getters

API level: Integration API

Signature

```
const std::vector<double>& getCamberDerivativeX() const noexcept;
const std::vector<double>& getCamberDerivativeY() const noexcept;
const std::vector<double>& getThicknessDerivativeX() const noexcept;
const std::vector<double>& getThicknessDerivativeY() const noexcept;
int getNumberOfCamberDerivativePoints() const noexcept;
int getNumberOfThicknessDerivativePoints() const noexcept;
```

Description

Return the stored imported derivative vectors and their support-point counts.

Notes

The count methods report the size of the corresponding x vector. They do not verify that the associated derivative vector has the same size.
These imported derivative fields are separate from the upper- and lower-surface derivative data calculated by `calculateGradientCurvature`.

15.7 Conformal-Mapping Data

The class can store auxiliary data generated by an external conformal-mapping component. `AirfoilData` does not calculate these fields itself in the shown implementation.

AirfoilData conformal-data setters

API level: Integration API

Signature

```
void setPointsConformalFunctionX(
        const std::vector<double>& values);
void setPointsConformalFunctionY(
        const std::vector<double>& values);
void setPointsConformalCircleX(
        const std::vector<double>& values);
void setPointsConformalCircleY(
        const std::vector<double>& values);
```

Description

Store externally generated conformal-function and conformal-circle data.

AirfoilData conformal-data setters *(continued)*

Parameters

`values`	Vector to copy into the selected conformal-mapping data field.

Notes

By library convention, the x field of the conformal-circle data may represent the angular parameter ϕ rather than a Cartesian coordinate.
The methods do not validate corresponding vector lengths.

AirfoilData conformal-data getters

API level: Integration API

Signature

```
const std::vector<double>& getConformalFunctionX() const noexcept;
const std::vector<double>& getConformalFunctionY() const noexcept;
const std::vector<double>& getConformalCircleX() const noexcept;
const std::vector<double>& getConformalCircleY() const noexcept;
int getNumberOfConformalFunctionPoints() const noexcept;
int getNumberOfConformalCirclePoints() const noexcept;
```

Description

Return the stored conformal-function and conformal-circle data and their support-point counts.

Notes

The count methods report the size of the corresponding x vector only. They do not establish dimensional consistency with the associated y vector.

15.8 Integration API Recommendations

When integrating `AirfoilData` into a larger application, the following practices are recommended:

- Use `setAirfoilPoints` instead of the independent `setPointsX` and `setPointsY` methods for ordinary coordinate input.
- Install an output callback when messages must be forwarded to a graphical user interface or central application logger.
- Ensure that the callback context remains valid for the complete lifetime of the callback registration.
- Use `setOutputSuppressed` only when messages are intentionally handled elsewhere. Continue checking all returned status values.
- Treat the independent vector setters as low-level operations. Restore all related dimensions and invariants before invoking a calculation method.
- Do not assume that a setter validates numerical finiteness unless this is explicitly stated.
- After replacing the airfoil contour or an intermediate distribution, recalculate all derived data that depend on it.
- Inspect `getConverged` before using externally stored aerodynamic values. Non-converged rows may contain placeholder values.

- Keep the distinction between externally supplied aerodynamic values and internally calculated inviscid geometric values:
  - `getAlpha0`, `getCl0`, `getCd0`, `getCm0`, `getClMax`, and `getEMax` return externally supplied aerodynamic scalar data,
  - `getAlpha0Geo` and `getCm0Geo` return values calculated internally from the camber line using thin-airfoil theory.
- Use the structure `AeroScalars` to supply the complete scalar aerodynamic data set to `setAerodynamicData`. Ensure that all six members are finite.
- Maintain consistent dimensions between pressure-coefficient matrices, pressure support coordinates, angles of attack, and convergence states.
- Supply all angles interpreted by the numerical methods of `AirfoilData` in degrees. This includes the angle passed to `rotate` and `calculateCirculation`. The angle returned by `normalizeAirfoil` and the value returned by `getAlpha0Geo` are also expressed in degrees. Imported aerodynamic angle data retain the convention of the supplying application; degrees are recommended.
- Use `setIsCreated(true)` only after the application considers the airfoil complete and ready for operations that require this state, such as `saveAirfoilCoordinates`.

16 Development and Contributing

Contributions to `AirfoilData` are welcome. They may include:

- corrections of numerical or geometric algorithms,
- additional input validation,
- improvements to numerical robustness,
- new airfoil-analysis methods,
- additional test scenarios,
- documentation corrections and extensions,
- portability improvements,
- improvements to the example and plotting scripts.

The repository contains a regression harness, sanitizer configurations, static-analysis targets, and coverage checks. Contributors should run the tests appropriate for their development environment before submitting a change.

Detailed repository-specific contribution rules are provided in:

```
CONTRIBUTING.md
```

This section explains the test architecture and the recommended development workflow.

16.1 Design Principles for Contributions

Contributions should preserve the design philosophy described in Section 1.1. A technically valid implementation may still be unsuitable for the library if it unnecessarily increases the integration effort, obscures the numerical procedure, or weakens existing validation and state guarantees.

The following principles apply to new contributions.

Keep Ordinary Use Simple

A common operation should remain possible without requiring users to understand internal helper classes or advanced C++ mechanisms.

Contributors should:

- preserve the short Basic API workflow,
- avoid introducing mandatory external dependencies without a compelling reason,
- avoid requiring generated files or a specialized build framework,
- prefer one clear public operation over several interdependent low-level calls,
- provide sensible defaults where they do not hide important physical assumptions.

If a new feature requires several configuration steps, consider whether a simple Basic API entry point and a more detailed Advanced API interface should both be provided.

Preserve Readability

Numerical source code should remain readable alongside the mathematical or engineering description.

Contributors should prefer:

- descriptive identifiers,
- explicit units in names or documentation,
- straightforward control flow,
- intermediate variables for physically meaningful quantities,
- comments explaining assumptions and numerical reasons,
- formulas written in an operation order that can be compared with the documented method.

Avoid introducing abstraction layers whose principal benefit is reducing the number of source lines. Reuse is desirable when it removes genuine duplication or centralizes an invariant, but not when it hides the numerical procedure.

Maintain the Small Dependency Footprint

The core library should remain usable with a conforming C++17 compiler and the C++ standard library.

A proposal for an additional mandatory dependency should explain:

- why the functionality cannot reasonably be implemented with the existing facilities,
- the effect on supported platforms,
- the effect on build and deployment complexity,
- the license compatibility,
- whether the dependency can remain optional.

Application-framework dependencies must not be introduced into the core classes. Framework-specific adaptation belongs in caller-side adapters.

Do Not Weaken Validation

New functionality must maintain or improve the existing defensive behavior.

Depending on the method, contributors should consider checks for:

- vector-size consistency,
- minimum and maximum point counts,
- valid support-point ordering,
- finite numerical input,
- finite intermediate and final results,
- division by zero or very small denominators,
- output-parameter aliasing,
- representable scaling and coordinate ranges,
- valid object initialization state.

A new success path should be accompanied by relevant failure-path tests.

Preserve Transactional State

Where a method can fail after performing several calculations, results should normally be prepared in local temporary storage and committed only after complete success.

Contributors should explicitly determine:

- which object members may change on success,
- which members must remain unchanged on failure,
- which previously derived data become stale,
- whether stale data must be cleared,
- whether a complete snapshot and rollback are required.

Partially updated geometry, coefficient vectors, or scalar results should not remain visible after a failed operation unless this behavior is unavoidable and explicitly documented.

Document Physical Meaning

Every new numerical output must have an unambiguous interpretation.

The documentation should state:

- whether the quantity is dimensional or normalized,
- the normalization used,
- the coordinate and sign conventions,
- the angle unit,
- the aerodynamic reference condition,
- whether the value is a physical result or a geometry descriptor,
- the underlying approximation or theory where relevant.

For example, the scaled source distribution and normalized circulation distribution require different scale factors and must not both be described only as generic “strength” values.

Add Focused Regression Tests

Every defect correction should include a scenario that would have detected the original defect. Every new public API should include at least one normal case and the relevant invalid or boundary cases.

The golden reference must be updated only after the numerical and semantic changes have been reviewed and accepted.

16.2 Source and Test Layout

The principal source and test files are organized as follows:

```
src/
    AirfoilData.h
    AirfoilData.cpp
    AkimaInterpolation.h
    AkimaInterpolation.cpp

tests/
      Makefile
      data/
           golden.txt
           NACA4618.dat
           NACA2312.dat
      src/
          harness_common.h
          harness_common.cpp
          golden_generator.cpp
          test_airfoildata.cpp
example.cpp
Makefile
plot/
     plot.gnu
```

The two core classes are located in the `src` directory. The test harness is located below `tests`. The documented example program is separate from the regression harness.

The example and the regression suite intentionally use different principal airfoils:

- NACA 2312 is used by the user-facing example,
- NACA 4618 is used by the principal golden-reference scenarios.

16.3 Test-Harness Architecture

The regression harness uses one common scenario corpus and two different output sinks.

The common scenarios are implemented in:

```
tests/src/harness_common.cpp
```

and are started by:

```
void runAllScenarios(HarnessSink& sink, const std::string& profilePath);
```

Each scenario emits tagged lines through the abstract `HarnessSink` interface:

```
class HarnessSink
{
    public:
    virtual ~HarnessSink() {}
    virtual void emit(const std::string& line) = 0;
};
```

Two programs use this common scenario corpus:

`golden_generator.cpp`
: Runs all scenarios and writes the emitted lines to the golden reference file.

`test_airfoildata.cpp`
: Runs the same scenarios and compares the generated results with the existing golden reference. Structural and textual fields are compared exactly, while serialized floating-point fields are interpreted numerically and compared using the configured tolerances.

This design ensures that the reference generator and the regression test execute exactly the same sequence of library calls. They differ only in how the generated output is handled.

16.4 Exact Serialization and Tolerance-Based Comparison

Floating-point values in the regression output are serialized with the C `%a` format:

```
std::snprintf(buffer, sizeof(buffer), "%a", value);
```

This hexadecimal representation preserves the exact binary value stored in a `double` without introducing an additional decimal-formatting error.

The regression harness does not require finite floating-point results to be bit-identical across all supported compilers and platforms. Small numerical differences can arise from:

- operating systems,
- compiler versions,
- GCC and Clang,
- mathematical-library implementations,
- processor architectures,
- permitted floating-point evaluation orders.

Each emitted record is first compared as an exact string. If both records are identical, the comparison succeeds immediately.

If the strings differ, the test program separates each record at its final `|` character. The complete prefix before this separator must match exactly. If both final fields can be parsed as floating-point values, the values are compared numerically.

For finite golden and actual values g and a, the comparison succeeds when:

$$|g - a| \leq 10^{-12} + 10^{-9} \max\left(|g|, |a|\right).$$

The configured tolerances are therefore:

$$\varepsilon_{\text{abs}} = 10^{-12}, \qquad \varepsilon_{\text{rel}} = 10^{-9}.$$

The absolute term controls comparisons near zero. The relative term controls comparisons at larger magnitudes.

Typical maximum accepted differences are:

$\max(\lvert g\rvert, \lvert a\rvert)$	Maximum accepted difference
0	1.0×10^{-12}
10^{-6}	1.001×10^{-12}
1	1.001×10^{-9}
10^{3}	1.000001×10^{-6}

Captured console records remain subject to exact textual comparison. The following information also remains exact:

- scenario tags,
- record structure,
- vector sizes,
- integer values,
- Boolean values,
- diagnostic text,
- record order,
- total record count.

NaN values are accepted only when their complete serialized records are identical. Infinite values must agree in classification and sign.

Note

The tolerance accommodates small platform-dependent floating-point differences. It is not intended to hide algorithmic regressions.
The tolerance values must not be increased merely to make a failing test pass. Any change requires a documented numerical justification and a review of the affected scenarios.

16.5 Scenario Coverage

The scenario corpus is intended to exercise every public method of the classes `AirfoilData` and `AkimaInterpolation` at least once.

The scenarios cover, among other cases:

- default object state,
- scalar and vector setters and getters,
- version information,
- callback output and console fallback,

- output suppression and diagnostic verbosity,
- desired point-count validation,
- support-point generation,
- leading-edge detection,
- scaling, rotation, and normalization,
- repair of chordwise contour reversals,
- camber, thickness, area, and nose calculations,
- derivative and second-derivative calculations,
- circulation and scaled source-distribution calculations,
- external aerodynamic data,
- pressure and conformal-mapping data storage,
- Akima interpolation and its derivatives,
- complete property calculation,
- airfoil redistribution,
- file output,
- empty and single-point input,
- inconsistent vector dimensions,
- duplicated and non-monotone support points,
- NaN and infinity,
- very large finite values,
- failed initialization and transactional rollback.

Private helper methods are covered indirectly through the public operations that invoke them.

The coverage target additionally reports which methods were executed by the test run. Method coverage does not by itself prove that all numerical paths or boundary cases have been tested; contributors should add focused scenarios for every corrected defect or newly introduced behavior.

16.6 Running the Tests During Development

For a normal GCC regression test, run:

```
make -C tests test-gcc
```

To test the same behavior with Clang, run:

```
make -C tests test-clang
```

The standard root-level test command is:

```
make test
```

This calls the complete `check` target in the test directory. A successful run ends with:

```
TEST OVERALL: PASSED
```

The complete check may require GCC, Clang, clang-tidy, gcov, and sanitizer support. If not all tools are installed, individual targets can be executed separately.

Sanitizer Checks

Run the GCC AddressSanitizer and UndefinedBehaviorSanitizer build with:

```
make -C tests san-gcc
```

Run the corresponding Clang build with additional implicit-conversion checks using:

```
make -C tests san-clang
```

Run the expanded GCC undefined-behavior checks with:

```
make -C tests san-full-gcc
```

Debug Standard-Library Checks

The following target enables the libstdc++ debug and assertion modes:

```
make -C tests dbg-gcc
```

This can detect invalid iterators, container misuse, and related boundary errors that may not be visible in an ordinary optimized build.

Coverage and Static Analysis

Run the gcov-based coverage check with:

```
make -C tests coverage
```

Run the GCC static analyzer with:

```
make -C tests analyzer
```

Run the Clang analyzer through clang-tidy with:

```
make -C tests tidy
```

Valgrind

If Valgrind is installed, run:

```
make -C tests memcheck
```

The combined extended check is:

```
make -C tests sharp
```

This includes the expanded sanitizer, debug, coverage, static-analysis, clang-tidy, and Valgrind checks.

16.7 Adding a Regression Scenario

A behavioral change, defect correction, or new public method should normally be accompanied by a scenario in:

```
tests/src/harness_common.cpp
```

A scenario is usually implemented as a small file-local function:

```
static void scenarioNewFeature(HarnessSink& sink)
{
    AirfoilData airfoil;

    // Prepare a deterministic input.
    std::vector<double> x = {1.0, 0.0, 1.0};
    std::vector<double> y = {-0.01, 0.0, 0.01};

    const bool result =
    airfoil.setAirfoilPoints(x, y);

    emitBool(
    sink,
    "newfeature.setpoints.ok",
    result);

    emitVector(
    sink,
    "newfeature.airfoilX",
    airfoil.getAirfoilX());
}
```

The scenario must then be called from `runAllScenarios`:

```
scenarioNewFeature(sink);
```

Use stable, descriptive tags. A recommended convention is:

```
<scenario>.<operation>.<result>
```

Examples are:

```
rotate.positive90.ok
rotate.positive90.x
rotate.positive90.y
akima.nonascending.init
aero.scalars.cm0
```

A new test should normally include:

- at least one successful call,
- the principal numerical or structural result,
- one or more relevant invalid inputs,
- confirmation that outputs or object state remain unchanged when the API promises transactional failure behavior.

For a defect correction, add a scenario that would have failed before the correction. This prevents the same defect from being introduced again.

16.8 Capturing Library Output

Some scenarios verify warnings, errors, or informational messages. The `ConsoleCapture` helper temporarily redirects both `std::cout` and `std::cerr` into one buffer:

```
{
    ConsoleCapture capture;

    const bool result =
    airfoil.someOperation();

    emitBool(
    sink,
    "operation.ok",
    result);

    emitConsole(
    sink,
    "operation.console",
    capture.text());
}
```

The captured text is split into individual tagged lines before it is passed to the sink.

When possible, tests of application output should also exercise the `PrintMethod` callback path. This verifies both the console fallback and the integration interface.

16.9 Updating the Golden Reference

The committed file:

```
tests/data/golden.txt
```

defines the expected structure, textual output, and numerical reference values of the documented library version. Textual and structural fields are compared exactly; finite floating-point values are compared within the configured absolute and relative tolerances.

A test failure can therefore indicate:

- a structural or textual difference,
- a missing or additional result,
- a vector-size or status difference,
- a floating-point deviation exceeding the accepted tolerance,
- an unexpected change between finite and non-finite values.

It must not be regenerated merely to make a failing test pass. A difference between actual output and the golden reference must first be classified as one of the following:

1. an unintended regression,
2. an intended numerical correction,
3. an intended API or diagnostic-message change,

4. a platform-specific difference requiring further investigation.

Only intended and reviewed changes should be incorporated into the golden reference.

To generate a new reference intentionally, run:

```
make -C tests golden-gcc
```

Then inspect the difference, for example with Git:

```
git diff -- tests/data/golden.txt
```

After reviewing the new reference, rerun at least:

```
make -C tests test-gcc
make -C tests test-clang
make -C tests san-gcc
make -C tests san-clang
```

Note

The normal regression target must compare against the existing committed golden reference. It should not regenerate the reference automatically before comparison, because that would convert an unintended result into the expected result.

16.10 Numerical Changes

Algebraically equivalent floating-point expressions can produce different binary results because operation order affects rounding.

When changing numerical code:

- determine whether the result is expected to remain bit-identical or only numerically equivalent,
- preserve documented operation order where it is significant,
- do not increase the regression tolerances merely to accept an unexplained difference,
- add a focused regression scenario,
- compare GCC and Clang results,
- compare Linux and Windows results where practical,
- inspect the magnitude and location of every changed numerical value,
- run the relevant sanitizer and debug-library targets,
- update the mathematical description if the numerical semantics changed.

Do not simplify or regroup expressions merely to reduce source length when the existing order is numerically significant.

Changes involving interpolation endpoints, support-point ordering, leading-edge detection, finite-difference stencils, Fourier coefficients, singularity-distribution normalization, geometric tolerances, or transactional object updates require particular care.

Note

A numerically tolerant comparison is not a license to reorder or simplify floating-point expressions without review. Small local differences can accumulate or become significant for extreme geometries.

16.11 Public API Changes

When adding or changing a public method, type, enumeration, or constant, update all relevant project components:

1. the declaration in the public header,
2. the implementation,
3. the scenario corpus,
4. the golden reference if behavior changed intentionally,
5. this user manual,
6. the example program if the preferred workflow changed,
7. `README.md` where appropriate,
8. `CHANGELOG.md`,
9. version information if required by the release policy.

A new public method should be assigned to one documentation level:

- **Basic API** for the preferred ordinary workflow,
- **Advanced API** for individual numerical processing steps,
- **Integration API** for application adapters and low-level data management.

The documentation should state:

- parameter units and coordinate conventions,
- required vector sizes and ordering,
- finite-value requirements,
- whether output parameters remain unchanged on failure,
- whether object state is modified transactionally,
- whether existing derived data are cleared,
- the exact meaning and normalization of returned numerical values.

16.12 Coding Conventions

The library uses C++17 and the C++ standard library. Contributions should follow the style of the surrounding source file.

The principal conventions are:

- use explicit and descriptive variable types in the numerical code,
- use `std::size_t` for vector sizes and indices where appropriate,
- use `enum class` instead of undocumented integer flags,
- mark important status returns with `[[nodiscard]]`,
- check numerical input with `std::isfinite` where required,

- avoid partially updated object state after failure,
- calculate results in local temporary variables or vectors and commit them only after complete success,
- preserve the correspondence between related x and y vectors,
- use comments to explain numerical or aerodynamic reasons, not only the syntax of the code,
- prefer straightforward control flow that remains readable to engineering and scientific users.

The core numerical implementation intentionally uses a mostly explicit C++ style. New uses of type deduction, generic templates, or lambda expressions should be limited to cases where they clearly improve a local implementation without making the numerical procedure harder to follow.

Warnings enabled by the test Makefile should be treated as development errors. The strict target can be run with:

```
make -C tests strict
```

16.13 Documentation and Theory

If a contribution changes the physical interpretation of a stored quantity, the documentation must be updated together with the source code.

This is particularly important for:

- angle units,
- normalized coordinate systems,
- source–sink and circulation distributions,
- derivative and curvature terminology,
- fixed aerodynamic reference conditions,
- distinctions between externally imported and internally calculated quantities.

The user manual should describe the API meaning without duplicating complete theoretical derivations. Detailed aerodynamic derivations should be cited from the relevant companion reports where appropriate.

16.14 Recommended Contribution Workflow

A typical contribution workflow is:

1. create a development branch,
2. implement one logically related change,
3. add or update the corresponding regression scenario,
4. build the example,
5. run the GCC and Clang regression tests,
6. run the relevant sanitizer and static-analysis targets,
7. inspect all numerical differences,
8. update the documentation and changelog,
9. update the golden reference only if the behavior change is intentional,
10. rerun the complete available test set,

11. submit the change for review.

Example commands are:

```
git checkout -b feature/my-change

make clean
make example
make -C tests test-gcc
make -C tests test-clang
make -C tests san-gcc
make -C tests san-clang
make -C tests coverage
```

If all required developer tools are installed, run:

```
make test
```

before submitting the contribution.

16.15 Contribution Checklist

Before submitting a change, verify the following points:

- The code compiles as C++17.
- GCC and Clang warnings have been reviewed.
- The ordinary Basic API workflow remains simple.
- No unnecessary mandatory dependency has been introduced.
- The implementation remains readable to engineering and scientific users.
- Important physical intermediate quantities have descriptive names.
- Units, normalizations, signs, and reference conditions are explicit.
- Input validation has not been weakened.
- NaN, infinity, vector dimensions, point counts, and support-point ordering are handled where relevant.
- Important returned status values are checked.
- Failure paths do not expose unintended partial results.
- Derived data invalidated by the change are cleared or recomputed.
- A focused regression scenario covers the intended behavior.
- Relevant invalid and boundary inputs are tested.
- GCC and Clang regression tests pass.
- Relevant sanitizer and static-analysis checks pass.
- The golden-reference difference has been reviewed.
- The Basic, Advanced, or Integration API classification is clear.
- The public API documentation matches the current header.
- The example and plot labels remain consistent with the API.
- `CHANGELOG.md` has been updated where appropriate.
- License and attribution requirements have been observed.
- Linux and Windows results have been compared where practical.
- Floating-point differences satisfy the documented absolute and relative tolerances.

- Test tolerances have not been increased without a documented numerical justification.
- NaN, infinity, signed infinity, and near-zero comparison behavior have been considered.
- Skipped optional checks have been reviewed and reported.
- Sanitizer checks have run successfully on at least one suitable environment for release-critical changes.

References

[1] O. Frommann, *Thin Airfoil Theory (Camber-Line Formulation) — Deep Documentation of a Teaching C++ Implementation*, Version 1.1.0, 2026. doi: 10.5281/zenodo.18422228. Available: https://github.com/of33/TAT.

[2] O. Frommann, *Thin Airfoil Theory (Thickness Formulation) — Deep Documentation of a Teaching C++ Implementation*, Version 1.0.0, 2026. doi: 10.5281/zenodo.19546291. Available: https://github.com/of33/TATT.